University Timetable Optimization and Management System — Complete Final Knowledge Document

Prepared from the complete UTOS chat context and uploaded project files

2026-04-26

University Timetable Optimization and Management System

Complete Final Knowledge Document

Project short name: UTOS
Full title: University Timetable Optimization and Management System
Document type: Super-deep project knowledge base + SRS + research plan + design document + implementation plan
Purpose: This document consolidates the project context, uploaded files, diagrams, requirements, use cases, research ideas, architecture, ERD notes, process model reasoning, solver planning, UI/UX planning, evaluation plan, risks, and future deep research roadmap into one self-explanatory Word document.

1. Document Purpose and Reading Guide

This document is designed as a single source of truth for the University Timetable Optimization and Management System. It is intentionally detailed and repetitive because it is meant to reduce cognitive load while preserving the complete project thinking. It can be used as a study document, report source, diagram guide, implementation planning base, and viva-preparation reference.

This document includes both polished explanations and raw documented knowledge. Some ideas are repeated in multiple forms: short form, formal form, beginner explanation, diagram explanation, implementation explanation, and research explanation. That repetition is intentional because the project is being learned step by step.

1.1 How to use this document

Use this document in layers:

1. Read the executive summary to understand the project quickly.
2. Read the problem, scope, and actors sections to understand what is being built.
3. Read the process model section for Assignment 1 and software-process justification.
4. Read the use case and requirements sections for Assignment 2 and SRS writing.
5. Read the diagrams section before drawing or correcting Visio diagrams.
6. Read the ERD/domain model section before database implementation.

7. Read the solver and constraint sections before algorithm implementation.
8. Read the deep research plan before doing literature review or final-year expansion.
9. Use the appendix as a raw memory base.

1.2 What this document is not

This document is not a minimal submission. It is not a short assignment. It is not only an SRS. It is not only a research plan. It is a full project knowledge base. A final classroom submission can be created later by selecting relevant sections and reducing repetition.

2. Source Inventory

This document uses and consolidates the materials present in the chat and uploaded file set. The uploaded raw knowledge base states that it is a text-first source of truth for assignments, SRS writing, diagrams, Visio recreation, implementation planning, and avoiding contradictions between use cases, DFDs, sequence diagrams, architecture, and database schema.

2.1 Uploaded and generated files available in the workspace

- Academic Timetable-2026-04-26-134435.png
- Assignment1_Process_Model_BSAI4B.docx
- ChatGPT Image Apr 26, 2026, 08_46_14 PM.png
- Drawing1.vsdx
- Drawing2.vsdx
- Executive Summary.pdf
- FF.docx
- Gemini_Generated_Image_90urf490urf490ur.png
- Gemini_Generated_Image_j1uj59j1uj59j1uj.png
- LICENSE
- Manus Installer.exe
- Microsoft.Services.Store.winmd
- README.md
- SRS eg KTH - Report(1).doc
- SRS.md
- SayedShabirProjectsoftware.pdf
- Screenshot 2026-04-21 215957.png
- Screenshot 2026-04-21 220025.png
- Screenshot 2026-04-25 184410.png
- Screenshot 2026-04-25 202001.png
- Screenshot 2026-04-25 231942.png
- Screenshot 2026-04-25 232441.png
- Screenshot 2026-04-25 235655.png

- Screenshot 2026-04-26 000113.png
- Screenshot 2026-04-26 002329.png
- Screenshot 2026-04-26 021453.png
- Screenshot 2026-04-26 022908.png
- Screenshot 2026-04-26 024101.png
- UTOS_ER_Diagram.drawio
- UTOS_ER_Diagram.mermaid
- UTOS_ER_Diagram.png
- UTOS_SE_Project_Document.docx
- UTOS_SE_Project_Document.pdf
- University_Timetable_Optimization_Knowledge_Base(3).jsonl
- University_Timetable_Optimization_Knowledge_Base(3).md
- University_Timetable_Optimization_Knowledge_Base(3).txt
- University_Timetable_Optimization_SRS_Super_Deep(1).docx
- University_Timetable_Optimization_SRS_Super_Deep(1).pdf
- V-model.jpg
- assignment1_process_model.html
- questioning the customers.docx

2.2 Important source categories

Core project documents

- UTOS_SE_Project_Document.docx
- UTOS_SE_Project_Document.pdf
- SRS.md
- University_Timetable_Optimization_SRS_Super_Deep(1).docx
- University_Timetable_Optimization_SRS_Super_Deep(1).pdf
- University_Timetable_Optimization_Knowledge_Base(3).md
- University_Timetable_Optimization_Knowledge_Base(3).txt
- University_Timetable_Optimization_Knowledge_Base(3).jsonl

Diagram and modeling files

- UTOS_ER_Diagram.png
- UTOS_ER_Diagram.drawio
- UTOS_ER_Diagram.mermaid
- Drawing1.vsdx
- Drawing2.vsdx
- Visio screenshots and diagram screenshots

Assignment and reference files

- Assignment1_Process_Model_BSAI4B.docx
- assignment1_process_model.html
- Executive Summary.pdf
- SayedShabirProjectsoftware.pdf
- SRS eg KTH - Report(1).doc
- V-model.jpg

3. Executive Summary

The University Timetable Optimization and Management System is a web-based platform that helps a university or department prepare weekly class timetables for teachers, student sections, courses, rooms, and timeslots. Instead of manually placing every class into a room and time, the administrator enters academic data, defines scheduling rules, selects policy preferences, and asks the system to generate a timetable draft. The system validates the data, checks conflicts, scores timetable quality, supports review and adjustment, handles approved change requests, repairs schedules with minimal disturbance, and publishes the approved timetable for teachers and students.

The project is not only a CRUD system. A CRUD-only system would store teachers, rooms, courses, and sections, but it would not solve the real problem: generating a valid and practical timetable under many interacting constraints. UTOS is better described as a constraint-based scheduling and decision-support system.

The first version should target one department or limited university scope. This keeps the project realistic while still being strong enough for a Software Engineering subject. The project includes master data management, hard constraints, soft preferences, generation, conflict reporting, manual adjustment, locking, change requests, re-optimization, timetable publication, and resource reports.

The selected process model is the Incremental Process Model. Prototyping is used as a supporting technique inside early increments, especially for screens, timetable grids, rule configuration, and reports. This is academically safer than calling the whole lifecycle a vague hybrid model. The system naturally divides into increments: data/calendar, constraints/preferences, generation, review/edit/lock, repair/re-optimization, and publishing/reports.

4. Project Identity

4.1 Formal project name

University Timetable Optimization and Management System.

4.2 Short project name

UTOS.

4.3 Simple definition

UTOS is a website that helps a university create class timetables automatically while avoiding clashes and respecting rules.

4.4 Strong definition

UTOS is an incrementally developed web-based university timetable optimization and management system that starts with reliable academic data, adds configurable hard constraints and soft preferences, generates and scores timetable drafts, supports review and manual locking, repairs schedules after approved changes with minimal disruption, and publishes final timetable views and reports for teachers, students, administrators, and facility managers.

4.5 Project type

- Software Engineering project.
- Web application.
- Scheduling system.
- Optimization system.
- Requirements and modeling project.
- Future implementation project.

4.6 What UTOS is

UTOS is a timetable generation system.

UTOS is a timetable management system.

UTOS is a timetable repair system.

UTOS is a timetable publishing system.

UTOS is a conflict detection system.

UTOS is a rule-based scheduling system.

UTOS is a decision-support system.

4.7 What UTOS is not

UTOS is not a full university ERP.

UTOS is not an attendance system.

UTOS is not a fee system.

UTOS is not a bus route system.

UTOS is not a hostel allocation system.

UTOS is not exam timetabling in the first version.

UTOS is not artificial intelligence prediction in the first version.

5. Problem Statement

5.1 Short problem statement

Manual university timetable creation is slow, error-prone, and difficult when many teachers, rooms, classes, student groups, holidays, and restrictions are involved. The system solves this by preventing teacher clashes, room clashes, section clashes, holiday violations, and room-capacity mismatches while making timetable quality visible and supporting controlled changes without rebuilding the whole schedule from zero.

5.2 Deep problem statement

University timetabling is a difficult practical scheduling problem. A university must schedule teachers, courses, student sections, rooms, and timeslots while avoiding conflicts and respecting institutional rules. A teacher cannot teach two classes at once. A room cannot host two classes at once. A student section cannot attend two classes at the same time. A class cannot be scheduled on a holiday. A large section should not be assigned to a small room. A lab class may require a lab room. A teacher may be unavailable at certain times. A department may prefer morning classes, early day ending, compact schedules, or nearby rooms.

A manually created timetable can easily contain hidden errors. Even if the timetable has no obvious clashes, it may still be low quality. It may create unnecessary gaps for students, overload teachers on one day, overuse a few rooms, schedule too many late classes, or move students across far buildings between consecutive classes. A strong timetable system should therefore not only check validity; it should also measure quality.

Another real-world problem is change. A teacher may request a change. A room may become unavailable. A holiday may be added. A department may reject the first draft. A good system should not destroy the whole accepted timetable every time a small change occurs. It should support controlled repair and minimal disturbance.

5.3 Problem categories

5.3.1 Data problem

Academic data is scattered. Teacher records, course records, room lists, section sizes, timeslots, and holidays may exist in separate files or informal communication. If this data is incomplete or wrong, the timetable will be wrong.

5.3.2 Constraint problem

Some rules are absolute, such as no teacher clash. Other rules are preferences, such as preferring earlier ending or nearby rooms. The system must separate hard constraints from soft preferences.

5.3.3 Optimization problem

There may be many possible valid timetables. The best timetable is not only valid; it also has good quality according to selected preferences.

5.3.4 Workflow problem

Timetabling is not one click. It includes data entry, validation, generation, review, adjustment, approval, publication, and later repair.

5.3.5 Change problem

A published timetable must sometimes be changed. The system should handle change requests and produce revised versions while minimizing unnecessary disruption.

6. Scope

6.1 In scope

The first version includes weekly course/class timetabling for one department, faculty, or limited university scope.

In scope:

- academic calendar management
- holiday management
- blocked-period management
- teacher management
- course management
- section management
- room management
- building/floor data
- timeslot management
- teacher availability
- room capacity and facilities
- hard constraint configuration
- soft preference configuration
- timetable generation
- conflict checking
- quality scoring
- manual adjustment by admin
- locking selected entries
- re-optimization after approved changes
- versioned timetable drafts
- published timetable versions

- teacher timetable view
- student section timetable view
- resource/room utilization reports
- export or print if time permits

6.2 Out of scope

Out of scope for the first version:

- exam timetabling
- bus route management
- transport route optimization
- hostel scheduling
- parking optimization
- attendance management
- fee management
- full ERP
- full multi-campus simulation
- personalized optimization for every individual student
- advanced AI prediction
- automatic replacement of academic policy decisions
- deep benchmarking inside the basic SRS

6.3 Scope boundary rule

UTOS manages the class timetable. It may consider traffic, energy, fatigue, room proximity, and teacher preferences as scheduling objectives, but it does not directly control buses, electricity systems, building access systems, or university policy.

7. Users, Actors, and Responsibilities

7.1 Timetable Administrator / Admin

The Timetable Administrator is the primary controlling actor. The Admin adds academic data, defines constraints, generates timetable drafts, reviews conflicts, manually adjusts entries, locks important assignments, approves final timetables, publishes the timetable, triggers re-optimization, views reports, and manages timetable versions.

Admin responsibilities

- Add courses.
- Add teachers.
- Add rooms.
- Add sections.
- Add timeslots.

- Add academic calendar.
- Add holidays.
- Define constraints.
- Define preferences.
- Start generation.
- Review timetable.
- Approve timetable.
- Publish timetable.
- Handle change requests.
- Re-optimize timetable.
- View reports.

7.2 Department Head / Department Coordinator

The Department Head or Department Coordinator is a supervisory actor. This actor checks whether the timetable matches department needs and policies. The coordinator may validate department data, review quality reports, request changes, and help decide preference weights.

7.3 Teacher

The Teacher is an operational actor. The teacher views personal timetable, submits availability or unavailability, and requests changes if needed. The teacher does not directly generate or publish the timetable.

7.4 Student

The Student is an end user. The student views the published section timetable. The student does not edit, generate, approve, or configure the timetable.

7.5 Facility Manager / Room Manager

The Facility Manager reviews room usage, free rooms, overused rooms, room capacity issues, and building load. This actor is important for operational reports.

7.6 System Administrator

The System Administrator manages users, roles, permissions, system settings, backups, and security controls. This actor supports the system but does not control academic scheduling logic.

8. Development Lifecycle and Process Model

8.1 Selected process model

The selected primary lifecycle is the Incremental Process Model. Prototyping is used only as a supporting validation technique inside selected early increments. This avoids the academic

weakness of inventing an unclear hybrid model. The process model is Incremental; prototyping is a technique.

8.2 Why Incremental fits UTOS

UTOS is modular. It can be built in controlled slices: master data, constraints, generation, review, repair, and publishing. Requirements may evolve after users see draft timetables. Soft-preference weights cannot be perfectly known before experimentation. The solver/generation feature is risky and should be tested early. Manual adjustment and re-optimization can be added later. Each increment can be tested separately. Coursework still requires documentation and traceability.

8.3 Why Waterfall is weak for UTOS

Waterfall assumes stable requirements. UTOS has hidden timetable preferences that may become clear only after stakeholders see a generated draft. Waterfall delays solver testing and user validation. Late teacher, room, or holiday changes would be expensive.

8.4 Why Spiral is not the best main model

Spiral is useful for large high-risk systems, but it is too heavy for a semester-scale academic project. UTOS has technical risk, but not safety-critical risk.

8.5 Why pure Agile/Scrum is not the formal answer

Agile practices can help, but the formal assignment requires clear process model documentation, diagrams, requirements, and traceability. Incremental development provides adaptability while remaining structured.

8.6 Increment plan

Increment	Focus	User-visible value
1	Master Data + Academic Calendar	Admin maintains teachers, courses, rooms, sections, timeslots, holidays, and blocked periods.
2	Constraints & Preferences	Admin configures hard constraints and weighted soft preferences.
3	First Working Timetable Generator	Admin generates a draft timetable and inspects conflicts or infeasibility.
4	Review, Scoring, Manual Editing & Locking	Admin reviews quality, edits entries, and locks decisions.
5	Repair / Re-optimization	Admin handles teacher/room/timing changes with minimal

Increment	Focus	User-visible value
		disruption.
6	Publishing + Role Views + Reports	Teachers and students view published timetables; Admin exports and reviews reports.

9. Requirements Specification

9.1 Functional requirements overview

The functional requirements are grouped around high-level use cases:

- UC-01 Maintain Scheduling Data
- UC-02 Generate Timetable
- UC-03 Review and Adjust Timetable
- UC-04 Publish and View Timetable
- UC-05 Manage Change Requests and Re-optimize

9.2 UC-01 Maintain Scheduling Data

The system shall allow Admin to add, update, view, and disable courses with course code, course name, credit hours, weekly session count, and department.

The system shall allow Admin to maintain teacher records with name, department, contact information, teaching load, and availability status.

The system shall allow Admin to maintain rooms with building, floor, capacity, room type, and available facilities such as lab equipment or projector.

The system shall allow Admin to maintain sections/student groups with program, semester, student count, and assigned courses.

The system shall allow Admin to define academic calendar dates, holidays, working days, and blocked periods.

9.3 UC-02 Generate Timetable

The system shall allow Admin to start timetable generation for a selected academic term and department.

The system shall prevent two classes from being assigned to the same room at the same timeslot.

The system shall prevent a teacher from being assigned to more than one class at the same timeslot.

The system shall prevent a student section from being assigned to more than one class at the same timeslot.

The system shall generate a draft timetable using active hard constraints and selected soft preferences.

The system shall display generation progress, completion status, and failure messages if no feasible timetable can be found.

9.4 UC-03 Review and Adjust Timetable

The system shall show the generated timetable in day-wise, teacher-wise, room-wise, and section-wise views.

The system shall display conflict counts, hard-constraint violations, soft-preference penalties, and room utilization summaries.

The system shall allow Admin to manually adjust selected timetable entries when permitted.

The system shall allow Admin to lock selected classes so that later re-optimization does not change them.

The system shall validate every manual change before saving it.

9.5 UC-04 Publish and View Timetable

The system shall allow Admin to publish only an approved timetable version.

The system shall allow Teachers to view their assigned teaching timetable.

The system shall allow Students to view the published timetable for their section, semester, or group.

The system shall allow users to filter the published timetable by day, course, teacher, room, section, or department.

The system shall support exporting the published timetable in printable or downloadable format.

9.6 UC-05 Manage Change Requests and Re-optimize

The system shall allow Teachers to submit timetable change requests with reason, affected class, preferred alternative time, and urgency.

The system shall allow the Department Head or Admin to approve, reject, or return change requests for clarification.

The system shall allow Admin to re-optimize the timetable after an approved change request while preserving locked and accepted entries where possible.

The system shall store each re-optimized timetable as a new version.

The system shall show the difference between the previous timetable version and the revised version before publication.

10. Non-Functional Requirements

10.1 Performance

The system should load normal pages quickly. Forms should save quickly. Timetable views should not be slow for department-scale data. The generation process may take longer than ordinary CRUD operations, but it should expose progress and not fail silently. Small prototype datasets should generate quickly, and infeasible results should be reported clearly.

10.2 Security

Only authorized users should log in. Admin should have timetable control. Teachers should see their own timetable and submit requests. Students should view only published timetable information. System Admin should manage accounts and roles. Passwords should be hashed. Production deployment should use HTTPS. Important actions should be logged.

10.3 Usability

The system should use clear forms, readable timetable grids, understandable error messages, visible conflict warnings, and simple filters. Admin should configure rules without editing source code. Teacher change-request forms should be simple. Student timetable view should be fast and easy.

10.4 Availability

The system should be available during university working hours and should allow published timetable viewing when the server is online. Data should persist in the database. Backups should be planned. Downtime should be minimized during timetable preparation and publishing periods.

10.5 Maintainability

The code should be modular. The solver should be separated from the backend. The database schema should support future extensions. New constraints should be added without rewriting the whole system.

10.6 Explainability

The system should explain why a timetable is invalid or low quality. It should show hard violations, soft penalties, unplaced classes, conflict messages, and score breakdown.

11. Use Case Knowledge

11.1 Compact use case set

Use this set for simple diagrams:

- UC-01 Maintain Scheduling Data

- UC-02 Generate Timetable
- UC-03 Review and Adjust Timetable
- UC-04 Publish and View Timetable
- UC-05 Manage Change Requests and Re-optimize

11.2 Expanded use case set

Use this set for a richer SRS:

- UC01 Maintain Academic Calendar
- UC02 Manage Master Data
- UC03 Define Constraints and Preferences
- UC04 Generate Timetable
- UC05 Review Timetable Quality
- UC06 Request Teacher or Room Change
- UC07 Re-optimize Timetable After Change
- UC08 Lock Manual Decisions
- UC09 Publish Timetable
- UC10 View Personal Timetable
- UC11 View Resource Reports
- UC12 Manage Users and Roles

11.3 Use case diagram rules

Actors are outside the system boundary. Use cases are inside the system boundary. A line between actor and use case means the actor triggers or participates in that use case. Teacher does not directly re-optimize the timetable; Teacher submits a change request. Student does not generate or edit timetable; Student views published timetable.

11.4 Compact actor connections

Actor	Use cases
Admin	UC-01, UC-02, UC-03, UC-04, UC-05
Department Head	UC-01, UC-03, UC-05
Teacher	UC-04, UC-05
Student	UC-04

12. Domain Model and ERD Knowledge

12.1 Current ERD entities

The current ERD contains these entities:

- TEACHERS
- COURSES

- ROOMS
- SECTIONS
- TIMESLOTS
- ACADEMIC_CALENDAR
- HOLIDAYS
- CONSTRAINTS
- TIMETABLE
- CHANGE_REQUESTS
- USERS

This is valid for a student-level SE project and directly supports the current SRS and diagrams.

12.2 Important modeling truth

A course is not always the thing being scheduled. A course can have multiple class sessions. A class session is the object that the solver should assign to a room and a timeslot. A timetable entry is the generated assignment result.

12.3 Improved future entities

For a stronger later implementation, add:

- DEPARTMENTS
- PROGRAMS
- ACADEMIC_TERMS
- COURSE_OFFERINGS
- CLASS_SESSIONS
- TEACHER_AVAILABILITY
- ROOM_FEATURES
- CONSTRAINT_SETS
- PREFERENCE_WEIGHTS
- SOLVER_RUNS
- TIMETABLE_VERSIONS
- TIMETABLE_ENTRIES
- VIOLATIONS
- MANUAL_LOCKS
- AUDIT_LOGS

12.4 ERD improvement recommendation

The current CONSTRAINTS table may appear isolated. Later, connect constraints to academic term, constraint set, solver run, or timetable version. This makes it clear which rules were active when a timetable was generated.

13. Constraint Model

13.1 Hard constraints

Hard constraints define feasibility. They must not be violated.

Examples:

- A teacher cannot teach two classes at the same timeslot.
- A room cannot host two classes at the same timeslot.
- A student section cannot attend two classes at the same timeslot.
- A class cannot be placed on a holiday.
- A class cannot be placed in a blocked period.
- A teacher cannot be assigned during unavailable time.
- A room must have enough capacity.
- A lab class must be assigned to a lab room if required.
- Locked timetable entries must not be changed during re-optimization.
- Every required class session must be scheduled or reported as unplaced.

13.2 Soft preferences

Soft preferences define quality. They can be violated with penalty.

Examples:

- Prefer early day ending.
- Prefer morning classes if selected.
- Prefer compact student schedules.
- Prefer compact teacher schedules.
- Prefer nearby rooms for consecutive student classes.
- Prefer same room for recurring sessions if room stability is selected.
- Prefer building compaction for energy-saving mode.
- Prefer reduced movement during peak traffic periods.
- Prefer lunch break or minimum break.
- Prefer balanced teacher load across days.
- Prefer minimizing changes during re-optimization.

13.3 Constraint documentation template

Field	Description
Rule name	Name of constraint or preference
Type	Hard or soft
Controlled by	Admin, Department Head, System policy
Input data needed	Data required to evaluate it

Field	Description
Logic	How the system checks it
Violation example	Example of failure
Penalty	Soft score penalty if applicable
Output message	What the system shows to user

14. Custom Policy Ideas

14.1 Continuous classing

The raw idea is to group classes close together to reduce idle time, room gaps, and possibly electricity use. The correct modeling decision is to make this a configurable soft preference. It should not become a universal hard rule because too many continuous classes may increase fatigue.

14.2 Energy saving

The raw idea is to use timetabling to reduce electricity or AC cost, especially in summer. This can be modeled as a soft preference that rewards using fewer active buildings, fewer scattered rooms, and appropriately sized rooms. Energy-aware course timetabling research supports the idea that classroom allocation can contribute to energy savings.

14.3 Traffic reduction

The raw idea is to reduce student movement and congestion. This can be modeled through room proximity, building transition penalties, and peak-time movement penalties. The system should not attempt full transport simulation in the first version.

14.4 Morning scheduling

Morning preference should be optional and configurable. Research on early morning university classes warns that early starts can be associated with lower attendance, shorter sleep, and poorer academic performance. Therefore, UTOS should not assume morning is always better.

14.5 Early ending

Early ending can be a soft preference. The system can penalize classes after a selected threshold and allow departments to configure the preferred latest end time.

14.6 Teacher changes without disturbing others

Teacher changes should be handled through change requests, approval flow, locked entries, minimal-disruption re-optimization, versioning, and difference reports.

15. DFD Knowledge

15.1 Level 0 DFD

The Level 0 DFD shows the whole system as one process. External entities are Admin, Department Coordinator, Teacher, Student, and Facility Manager. Data flows include master data, constraints, generation command, generated timetable, conflict report, availability, change request, personal timetable, section timetable, report request, and room utilization report.

15.2 Level 1 DFD

Processes:

1. Manage Master Data
2. Configure Constraints
3. Generate Timetable
4. Handle Change Requests
5. Publish & View Timetable
6. Generate Reports

Data stores:

- D1 Master Data
- D2 Constraints & Preferences
- D3 Timetable Versions
- D4 Change Requests

15.3 Level 2 DFD

Level 2 decomposes Process 3 Generate Timetable:

- 3.1 Validate Input
- 3.2 Build Model
- 3.3 Run Solver
- 3.4 Score Conflicts
- 3.5 Save Timetable

Flow:

D1 and D2 feed into Validate Input. Validated data becomes the problem model. The solver produces raw assignment. Score Conflicts produces a scored timetable and conflict report. Save Timetable stores the final timetable in D3.

16. Swimlane Workflow

16.1 Swimlane lanes

- Admin
- System
- Teacher / Student

16.2 Initial generation flow

1. Admin enters academic data.
2. Admin defines constraints and preferences.
3. Teacher submits availability and preferences.
4. System validates input data.
5. Admin starts timetable generation.
6. System runs scheduling engine.
7. System checks conflicts and optimizes.
8. System generates draft timetable.
9. Admin reviews draft.
10. Admin approves or adjusts constraints/data.
11. Admin publishes timetable.
12. System saves final timetable.
13. System notifies users.
14. Teachers/students view timetable.

16.3 Change request flow

1. Teacher views timetable.
2. Teacher submits change request.
3. System records request.
4. Admin or System reviews request.
5. Decision: approved or rejected.
6. If rejected, user receives status.
7. If approved, system updates or re-optimizes timetable.
8. System sends updated timetable.
9. Teacher/student receives updated timetable.

17. System Sequence Diagram Knowledge

Use case: Generate Timetable.

Lifelines: Admin and System.

Sequence:

1. Admin selects Generate Timetable.

2. System displays generation form.
3. Admin enters semester, department, sections, and rules.
4. System shows selected data summary.
5. Admin confirms generation.
6. System shows progress indicator.
7. System validates input data.
8. System checks rooms, teachers, timeslots, holidays.
9. System generates timetable.
10. System checks clashes/conflicts.
11. System displays generated timetable.
12. System displays conflicts/warnings.
13. Admin reviews timetable.
14. Admin saves timetable as draft.
15. System confirms draft saved successfully.

18. Architecture

18.1 Components

- Web Frontend
- Backend API
- Database
- Solver Engine

18.2 Component responsibilities

Web Frontend

The Web Frontend runs in the user browser. It displays forms, dashboards, timetable grids, conflict reports, change request forms, and published timetable views.

Backend API

The Backend API handles business logic, authentication, validation, workflow, database access, and solver orchestration.

Database

The Database stores users, roles, teachers, courses, rooms, sections, timeslots, holidays, constraints, timetable versions, timetable entries, change requests, violations, and audit logs.

Solver Engine

The Solver Engine receives a problem model and returns timetable assignments, unplaced classes, conflict reports, and quality scores.

18.3 Data flow

Frontend sends HTTPS/REST requests to Backend.

Backend sends JSON responses to Frontend.

Backend sends SQL queries to Database.

Database returns data records to Backend.

Backend sends problem model to Solver.

Solver returns assignment result to Backend.

Backend saves timetable version in Database.

Backend returns timetable and report to Frontend.

18.4 Architecture rule

Frontend does not talk directly to Database. Frontend does not talk directly to Solver. Backend is the orchestrator. Solver should be replaceable.

19. Solver and Algorithm Research

19.1 Solver input

The solver receives teachers, courses, course offerings, class sessions, sections, rooms, room features, buildings, timeslots, teacher availability, holidays, blocked periods, hard constraints, soft preferences, preference weights, locked entries, and existing timetable version if repairing.

19.2 Solver output

The solver returns timetable entries, room assignments, timeslot assignments, unplaced classes, conflict report, hard violation count, soft penalty score, quality score, runtime, and solver status.

19.3 Solver pipeline

1. Validate input.
2. Build problem model.
3. Run solver.
4. Score conflicts.
5. Save timetable.
6. Return result.

19.4 Candidate algorithms

- Greedy heuristic
- Backtracking

- Constraint programming
- CP-SAT
- Genetic algorithm
- Simulated annealing
- Tabu search
- Large neighborhood search
- Hybrid heuristic
- Minimal perturbation repair

19.5 MVP algorithm strategy

Start with a simple greedy or constructive heuristic. Add conflict detection. Add soft scoring. Only after the data model and workflow are stable, move toward CP-SAT or a more advanced solver.

20. Research Literature and Evidence Base

20.1 Curriculum-Based Course Timetabling

The ITC-2007 Curriculum-Based Course Timetabling formulation is a strong academic base. It defines weekly assignment of university course lectures to rooms and time periods, with hard constraints and soft constraints. Bettinelli et al. describe CB-CTT as finding the best weekly assignment of lectures to rooms and periods, where feasible schedules satisfy hard constraints and minimize soft-constraint penalties.

20.2 UniTime

UniTime is useful as a real-world course timetabling system reference. Its documentation explains loading input data, checking data consistency, searching for complete timetables, reviewing properties, viewing timetables, making changes, saving timetables, and using an interactive solver.

20.3 OR-Tools CP-SAT

Google OR-Tools CP-SAT is a strong future implementation candidate because scheduling can be represented with variables and constraints. Teachers, rooms, and sections can be modeled as resources with no-overlap constraints.

20.4 Energy-aware timetabling

Energy-aware course timetabling research supports the idea that classroom allocation and timetable structure can influence building energy use. This supports UTOS energy-saving soft preference, but not as a universal hard rule.

20.5 Early morning class caution

Research on early morning university classes shows that early starts can be associated with lower attendance, shorter sleep, and poorer academic outcomes. UTOS should therefore make morning preference configurable rather than treating morning classes as automatically superior.

20.6 Minimal perturbation / repair

Minimal perturbation research supports the repair workflow. After a change request, the system should revise the timetable while keeping the new timetable close to the original.

21. Deep Research Plan

21.1 Research goal

The goal of deep research is to transform UTOS from a good SE documentation project into a research-informed, implementation-ready timetable optimization system.

21.2 Phase 1 — Source consolidation

Collect all uploaded project files, diagrams, screenshots, SRS drafts, assignments, and knowledge base files. Catalogue them by purpose: requirements, process model, diagrams, research, ERD, reference examples, and implementation planning.

Deliverable: source inventory and consistency map.

21.3 Phase 2 — Literature review

Research:

- ITC-2007 Curriculum-Based Course Timetabling
- university course timetabling surveys
- UniTime documentation
- OR-Tools CP-SAT scheduling
- energy-aware timetabling
- campus movement / room proximity studies
- minimal perturbation / timetable repair
- early morning class studies

Deliverable: annotated bibliography.

21.4 Phase 3 — Formal problem definition

Define:

- exact problem variant
- scope size

- entities
- planning variables
- hard constraints
- soft constraints
- objective function
- repair objective
- evaluation metrics

Deliverable: formal problem statement.

21.5 Phase 4 — Data model refinement

Improve the ERD by adding course offerings, class sessions, timetable versions, solver runs, violations, locks, and audit logs.

Deliverable: improved ERD and data dictionary.

21.6 Phase 5 — Solver experiments

Run experiments with:

- greedy solver
- greedy + local improvement
- CP-SAT small model
- repair model with locked entries

Measure feasibility, runtime, hard violations, soft penalties, and number of changes after repair.

Deliverable: solver experiment report.

21.7 Phase 6 — UI/UX research

Design and test:

- master data forms
- constraint configuration screen
- generation screen
- conflict report
- timetable grid
- manual edit screen
- change request workflow
- reports

Deliverable: UI wireframes and usability notes.

21.8 Phase 7 — Evaluation plan

Define success metrics:

- hard violations = 0
- minimized soft penalty
- generation time
- room utilization
- teacher load balance
- student gaps
- late class count
- room proximity score
- number of changed entries during repair

Deliverable: evaluation matrix.

21.9 Phase 8 — Implementation roadmap

Turn research into build plan with increments, stack, database schema, APIs, UI screens, solver integration, tests, deployment, and final report.

Deliverable: implementation blueprint.

22. Implementation Roadmap

22.1 Foundation first

Do not start with the algorithm first. Start with clean data, database, CRUD, validation, and conflict checker.

22.2 Step-by-step plan

1. Create repository.
2. Create backend project.
3. Create database.
4. Create users table.
5. Create teachers table.
6. Create courses table.
7. Create rooms table.
8. Create sections table.
9. Create timeslots table.
10. Create academic calendar table.
11. Create holidays table.
12. Create constraints table.
13. Create timetable versions.

14. Create timetable entries.
15. Create change requests.
16. Build master data forms.
17. Build constraint screen.
18. Build teacher availability screen.
19. Build generate timetable button.
20. Build validation function.
21. Build conflict checker.
22. Build simple generator.
23. Build timetable grid.
24. Build conflict report.
25. Build manual edit.
26. Build lock/unlock.
27. Build change request.
28. Build re-optimization.
29. Build publish.
30. Build teacher/student view.
31. Build reports.
32. Build export.
33. Test.
34. Document.
35. Present.

23. Evaluation Metrics

23.1 Feasibility metrics

- hard violations count
- teacher conflicts count
- room conflicts count
- section conflicts count
- holiday violations count
- capacity violations count
- unplaced sessions count

23.2 Quality metrics

- soft penalty score
- room utilization
- teacher load balance
- student gap count
- late class count
- morning preference satisfaction

- room proximity score
- building movement score
- energy compaction score
- room stability score
- minimum working days score

23.3 Performance metrics

- generation runtime
- conflict checking runtime
- timetable loading time
- report loading time
- database query time
- number of sessions scheduled
- number of rooms used
- number of timeslots used

23.4 Repair metrics

- number of changed entries
- number of locked entries preserved
- number of affected teachers
- number of affected sections
- distance from original timetable
- change request resolution time

24. Testing Plan

24.1 Unit tests

Test teacher conflict detection.

Test room conflict detection.

Test section conflict detection.

Test room capacity check.

Test holiday check.

Test timeslot overlap.

Test lock preservation.

Test change request status.

24.2 Integration tests

Test frontend form to backend.

Test backend to database.

Test backend to solver.

Test solver result to database.

Test publish to teacher view.

Test publish to student view.

Test change request to re-optimization.

24.3 System tests

Generate timetable with small dataset.

Generate timetable with medium dataset.

Generate timetable with infeasible dataset.

Generate timetable with holidays.

Generate timetable with unavailable teacher.

Generate timetable with limited rooms.

Generate timetable with locked entries.

Generate timetable after change request.

24.4 Acceptance tests

Admin can maintain scheduling data.

Admin can configure constraints.

Admin can generate timetable.

Admin can review conflicts.

Admin can publish timetable.

Teacher can view timetable.

Student can view timetable.

Teacher can request change.

Admin can process change.

Facility Manager can view reports.

25. UI Screen Plan

Screens:

- Login screen
- Dashboard
- Teacher management screen
- Course management screen
- Room management screen
- Section management screen
- Timeslot management screen
- Academic calendar screen
- Holiday screen
- Constraint configuration screen
- Preference weight screen
- Teacher availability screen
- Generate timetable screen
- Solver progress screen
- Conflict report screen
- Timetable grid screen
- Manual edit screen
- Lock/unlock screen
- Change request form
- Change request review screen
- Version comparison screen
- Publish screen
- Teacher timetable view
- Student timetable view
- Room utilization report
- Export screen
- User management screen

26. Risk Register

Risk	Probability	Impact	Mitigation
Incomplete input data	High	High	Add validation and warnings before generation.
Solver too complex	Medium	High	Start with simple heuristic, later improve.
Scope creep	High	High	Keep out-of-scope

Risk	Probability	Impact	Mitigation
			list visible.
Conflicting soft preferences	High	Medium	Use weights and explain trade-offs.
Teacher change after publication	Medium	High	Use change request and re-optimization.
Poor UI usability	Medium	Medium	Prototype screens early.
Diagrams inconsistent	Medium	Medium	Use this knowledge base as source of truth.
Algorithm not feasible for large data	Medium	High	Test small dataset first, then scale.

27. Diagram Appendix

This section documents the diagram/image artifacts included in the chat. For performance and reliability, this final Word document lists the source image names and explains their purpose. The original image files remain available separately in the upload set.

- Figure artifact 1: UTOS_ER_Diagram.png stored as fig_01.png in the generated asset folder.
- Figure artifact 2: V-model.jpg stored as fig_02.jpg in the generated asset folder.
- Figure artifact 3: Academic Timetable-2026-04-26-134435.png stored as fig_03.png in the generated asset folder.
- Figure artifact 4: ChatGPT Image Apr 26, 2026, 08_46_14 PM.png stored as fig_04.png in the generated asset folder.
- Figure artifact 5: Gemini_Generated_Image_90urf490urf490ur.png stored as fig_05.png in the generated asset folder.
- Figure artifact 6: Gemini_Generated_Image_j1uj59j1uj59j1uj.png stored as fig_06.png in the generated asset folder.
- Figure artifact 7: Screenshot 2026-04-25 184410.png stored as fig_07.png in the generated asset folder.
- Figure artifact 8: Screenshot 2026-04-25 202001.png stored as fig_08.png in the generated asset folder.
- Figure artifact 9: Screenshot 2026-04-25 231942.png stored as fig_09.png in the generated asset folder.
- Figure artifact 10: Screenshot 2026-04-25 232441.png stored as fig_10.png in the generated asset folder.

- Figure artifact 11: Screenshot 2026-04-25 235655.png stored as fig_11.png in the generated asset folder.
- Figure artifact 12: Screenshot 2026-04-26 000113.png stored as fig_12.png in the generated asset folder.
- Figure artifact 13: Screenshot 2026-04-26 002329.png stored as fig_13.png in the generated asset folder.
- Figure artifact 14: Screenshot 2026-04-26 021453.png stored as fig_14.png in the generated asset folder.
- Figure artifact 15: Screenshot 2026-04-26 022908.png stored as fig_15.png in the generated asset folder.
- Figure artifact 16: Screenshot 2026-04-26 024101.png stored as fig_16.png in the generated asset folder.
- Figure artifact 17: Screenshot 2026-04-21 215957.png stored as fig_17.png in the generated asset folder.
- Figure artifact 18: Screenshot 2026-04-21 220025.png stored as fig_18.png in the generated asset folder.

28. References and Research Sources

The following sources are important for the deep research version of UTOS:

1. Bettinelli, A., Cacchiani, V., Roberti, R., and Toth, P. “An overview of curriculum-based course timetabling.” TOP, 2015. https://link.springer.com/article/10.1007/s11750-015-0366-z
2. ITC-2007 Curriculum-Based Course Timetabling Track 3 report. https://www.researchgate.net/publication/215777351_The_Second_International_Timetabling_Competition_ITC-2007_Curriculum-based_Course_Timetabling_Track_3
3. UniTime Course Timetabling documentation. https://help.unitime.org/course-timetabling
4. UniTime Course Timetabling Solver documentation/manual. https://help.unitime.org/course-timetabling-solver and https://help.unitime.org/manuals/courses-solver
5. Google OR-Tools CP-SAT scheduling documentation. https://github.com/google/or-tools/blob/stable/ortools/sat/docs/scheduling.md
6. Song et al. “Energy efficiency-based course timetabling for university buildings.” Energy, 2017. https://www.sciencedirect.com/science/article/pii/S0360544217313701
7. Yeo et al. “Early morning university classes are associated with impaired sleep and academic performance.” Nature Human Behaviour, 2023. https://www.nature.com/articles/s41562-023-01531-x
8. UniTime minimal perturbation / course timetabling repair papers. https://www.unitime.org/papers/patat05.pdf

29. Viva Preparation

29.1 What is your project?

My project is a University Timetable Optimization and Management System. It is a web-based system that manages academic data and generates weekly class timetables using hard constraints and soft preferences.

29.2 Why is it not just CRUD?

Because the system does not only store data. It uses stored data to generate valid timetable assignments, detect conflicts, calculate quality scores, support manual repair, and publish role-based timetable views.

29.3 Why Incremental Model?

Because UTOS can be built in meaningful modules: data management, constraints, generation, review, repair, and publishing. Timetable preferences become clearer after users see draft timetables.

29.4 What is the hardest part?

The hardest part is constraint modeling and solver logic because many rules interact: teacher availability, room capacity, section conflicts, holidays, soft preferences, and change repair.

29.5 What is re-optimization?

Re-optimization means repairing an existing timetable after an approved change while keeping the revised timetable as close as possible to the previous version.

30. Raw Appendix: Full Existing Knowledge Base

The following appendix preserves the raw text knowledge base content. It is intentionally repetitive and highly detailed.

University Timetable Optimization System — Raw Text Knowledge Base

Project short name: UTOS / University Timetable Optimization and Management System
Knowledge base type: raw documented project knowledge, assignment knowledge, requirements knowledge, and diagram-building guide
Generated: 2026-04-26 17:30
Purpose: keep all project facts in one text-first file so it can be pasted into Claude/ChatGPT, used as report source material, or used as a reference while recreating diagrams in Microsoft Visio.

---

0. How to Use This Knowledge Base

This file is not written like a final submission report. It is a reusable knowledge base.

Use it for: - writing Assignment 1: Process Model Selection; - writing Assignment 2: High-Level Use Cases; - building the SRS-style project document; - recreating diagrams in Visio; - feeding another AI tool with clear project facts; - planning implementation after documentation; - avoiding repeated contradictions between use cases, DFDs, sequence diagrams, architecture, and database schema.

Important rule: - Treat this as the raw source of truth. - When writing final assignments, convert these points into formal paragraphs. - When drawing diagrams, do not copy every possible feature into one diagram. Use the diagram-specific sections.

PART A — PROJECT IDENTITY

A1. Project Name

University Timetable Optimization and Management System.

Possible short names: - University Timetable Optimization System - UTOS - Optimize Flow

Best formal title: **University Timetable Optimization and Management System**

A2. One-Paragraph System Description

The system is a web-based platform that helps a university or department create, manage, improve, and publish weekly class timetables for teachers, student sections, courses, rooms, and timeslots. Instead of manually placing every class into a room and time, the administrator enters academic data, defines scheduling rules, selects policy preferences, and requests timetable generation. The system validates the data, generates a draft timetable, checks conflicts, scores timetable quality, supports manual review and controlled adjustment, repairs schedules after approved changes, and publishes the approved timetable for teachers and students.

A3. Short Problem Statement

Manual university timetable creation is slow, error-prone, and difficult when many teachers, rooms, student sections, holidays, and restrictions are involved. The system solves this by preventing teacher clashes, room clashes, section clashes, holiday violations, and room-capacity mismatches while also making timetable quality visible and supporting controlled changes without rebuilding the whole schedule from zero.

A4. Core Project Statement

A web-based university timetable optimization and management system that generates, customizes, reviews, repairs, and publishes weekly class schedules under hard constraints and configurable soft preferences such as teacher availability, room suitability, room proximity, compact scheduling, early day ending, energy-aware grouping, reduced student movement, and minimal disruption after changes.

A5. Immediate Academic Purpose

The project is for a Software Engineering subject. The required documentation style includes: - a process model selection assignment; - high-level use cases with essential and real descriptions; - software requirements; - use case diagram; - system sequence diagram; - architecture diagram; - other modelling diagrams such as DFD and swimlane workflow; - Visio-based diagrams.

PART B — SCOPE

B1. In Scope for First Version

The first version should include: - weekly course/class timetabling for one department, faculty, or limited university scope; - academic calendar and holiday handling; - courses, teachers, sections, rooms, buildings, and timeslots; - teacher availability and unavailable periods; - room capacity and room facilities; - hard constraints; - soft preferences; - timetable generation; - conflict checking; - timetable quality scoring; - manual adjustment by admin; - locking selected timetable entries; - re-optimization after approved changes; - versioned timetable drafts and published timetable versions; - teacher timetable view; - student section timetable view; - reports for room utilization and timetable quality; - export/print of final timetable if time permits.

B2. Explicitly Out of Scope for First Version

Do not include these in the first version: - exam timetabling; - bus route management; - transport route optimization; - hostel scheduling; - parking optimization; - attendance management; - fee management; - full university ERP; - full multi-campus simulation; - personalized timetables for every individual student; - advanced AI prediction; - automatic replacement of human academic policy decisions; - deep algorithm comparison and benchmarking inside the basic SRS.

B3. Scope Boundary Rule

The system manages the **class timetable**, not the whole university operation. It may consider traffic, energy, fatigue, room proximity, and teacher preferences as scheduling objectives, but it does not directly control buses, electricity systems, building access systems, or academic policy.

PART C — USERS, ACTORS, AND RESPONSIBILITIES

C1. Main Actors

C1.1 Timetable Administrator / Admin

Primary controlling actor. Needs: - add and update academic data; - define constraints and preferences; - generate timetable drafts; - review conflicts and scores; - manually adjust timetable entries; - lock important assignments; - approve final timetable; - publish timetable; - trigger re-optimization after changes; - view reports; - manage timetable versions.

C1.2 Department Head / Department Coordinator

Supervisory/supporting actor. Needs: - validate department data; - review whether generated timetable suits department policy; - request timetable changes; - approve or reject change requests depending on local policy; - review timetable quality and policy effects; - help decide preference weights such as morning classes, early finish, traffic reduction, or energy-saving mode.

C1.3 Teacher

Operational actor. Needs: - view personal teaching timetable; - submit availability or unavailability information; - request timetable change due to leave, clash, room issue, or preference; - see whether approved changes affected their timetable.

C1.4 Student

End-user actor. Needs: - view official timetable for section, semester, or student group; - see course, teacher, room, day, and time; - access only published timetable, not drafts.

C1.5 Facility Manager / Room Manager

Resource actor. Needs: - maintain or validate room information; - check room capacity and room usage; - see free, used, underused, and overused rooms; - review building load or room utilization; - identify rooms unsuitable for certain classes.

C1.6 System Administrator

Technical admin actor. Needs: - manage user accounts; - manage roles and permissions; - configure system settings; - support backups; - monitor security and access control.

C2. Actor Boundary Rules for Use Case Diagrams

- Actors must be outside the system boundary.
- Use cases must be inside the system boundary.
- A line between actor and use case means the actor triggers or participates in that use case.
- The Admin should not be shown as inside the system.
- The database, solver, and backend are internal components, not actors in the basic use case diagram.

- Teacher does not directly re-optimize the timetable; Teacher submits a change request.
- Student does not generate or edit timetable; Student views the published timetable.
- Department Head can review and approve but does not usually perform low-level solver operation.
- Facility Manager interacts mainly with room/resource reports or room data.

PART D — SYSTEM DEPENDENCIES

D1. Required Dependencies

The system depends on: - web browser; - internet or local university network; - frontend web interface; - backend server/API; - database; - authentication mechanism; - solver/optimization engine; - university-provided academic data.

D2. Important Internal Components

Four main architecture components: 1. Frontend 2. Backend API 3. Database 4. Solver Engine

D3. Optional Dependencies

Optional later dependencies: - email notification service; - SMS or push notifications; - PDF export tool; - Excel/CSV export; - university ERP integration; - single sign-on; - external room booking system.

D4. Dependency Rule for Architecture Diagram

The architecture diagram must show system components and data flow, not human workflow. Correct component flow: - Frontend sends HTTPS/REST requests to Backend API. - Backend API reads/writes Database using SQL queries. - Backend API sends problem model to Solver Engine. - Solver Engine returns assignment result, score, infeasibility reasons, and warnings. - Backend stores timetable version in Database. - Frontend displays timetable, conflicts, scores, and reports.

PART E — DOMAIN CONCEPTS AND GLOSSARY

E1. Key Terms

Admin: - timetable administrator who operates the scheduling system.

Academic Term: - semester or academic session for which timetable is generated.

Academic Calendar: - working days, semester dates, holidays, blackout periods, special closure days, exam weeks.

Course: - academic subject/course to be scheduled.

Course Offering: - a course instance offered in a specific term, department, program, or section.

Section: - student group/cohort taking a set of courses in a semester.

Class Session: - one scheduled meeting of a course, e.g., lecture, lab, tutorial.

Teacher: - instructor assigned to one or more courses or class sessions.

Room: - physical classroom, lecture hall, or lab.

Building: - location containing rooms, possibly with floor and distance metadata.

Timeslot: - day and time range in which a class can be scheduled.

Hard Constraint: - rule that must not be violated, e.g., no teacher clash.

Soft Preference: - desirable goal that may be violated with penalty, e.g., early finish.

Constraint Weight: - numeric importance assigned to a soft preference.

Solver Engine: - optimization component that assigns class sessions to timeslots and rooms.

Timetable Entry: - one scheduled assignment linking course/session, teacher, section, room, timeslot, and version.

Timetable Version: - saved draft or published copy of timetable.

Manual Lock: - fixed timetable entry that future re-optimization must preserve.

Change Request: - request from teacher/department to change a class, room, or time.

Re-optimization: - repairing an existing timetable after a change while minimizing unnecessary disturbance.

E2. Main Domain Entities

Minimum entity list: - User - Role - Department - Program - AcademicTerm - AcademicCalendar - Holiday - BlackoutPeriod - Course - CourseOffering - Section - Teacher - TeacherAvailability - Room - Building - RoomFeature - Timeslot - ConstraintRule - PreferenceWeight - TimetableVersion - TimetableEntry - ManualLock - ChangeRequest - AuditLog - Report

E3. Core Entity Relationships

- Department has many Programs.
- Program has many Sections.
- AcademicTerm has one AcademicCalendar.
- AcademicCalendar has many Holidays and BlackoutPeriods.
- Department offers many Courses.
- Course can have many CourseOfferings.
- CourseOffering is assigned to one or more Sections.

- Teacher teaches CourseOfferings or ClassSessions.
- Room belongs to Building.
- Room has capacity and facilities.
- Timeslot defines possible scheduling windows.
- TimetableVersion contains many TimetableEntries.
- TimetableEntry links CourseOffering/ClassSession + Teacher + Section + Room + Timeslot.
- ChangeRequest references a TimetableEntry or affected class/session.
- ManualLock protects a TimetableEntry from automatic modification.
- AuditLog records generation, edits, locks, approvals, and publication.

PART F — CONSTRAINT MODEL

F1. Hard Constraints

Hard constraints must not be violated. Examples: - A teacher cannot teach two classes at the same timeslot. - A room cannot host two classes at the same timeslot. - A student section cannot attend two classes at the same timeslot. - A class cannot be placed on a holiday. - A class cannot be placed in a blocked period. - A teacher cannot be assigned during unavailable time. - A room must have capacity equal to or greater than student count. - A lab course must be assigned to a lab room if required. - Locked timetable entries must not be changed during re-optimization. - Published timetable cannot be modified directly without creating a new version or revision.

F2. Soft Preferences

Soft preferences are desirable but can be violated with penalty. Examples: - Prefer early day ending. - Prefer morning classes if selected. - Prefer compact student schedules. - Prefer compact teacher schedules. - Prefer nearby rooms for consecutive student classes. - Prefer same room for recurring sessions if room stability is selected. - Prefer different rooms occasionally if room variety is selected. - Prefer building compaction for energy-saving mode. - Prefer reduced movement during peak traffic periods. - Prefer lunch break or minimum break. - Prefer balanced teacher load across days. - Prefer avoiding too many consecutive classes to reduce fatigue. - Prefer minimizing changes during re-optimization.

F3. User's Custom Policy Ideas Converted to System Rules

Energy-saving continuous classing

Raw idea: - Schedule classes continuously or in selected room/building clusters to reduce electricity/AC usage.

Correct system interpretation: - Model as a configurable soft objective, not a universal hard rule. - Penalize scattered room usage if energy mode is enabled. - Reward grouping classes in fewer buildings/rooms during selected periods. - Do not force unhealthy continuous classes for students or teachers.

Circular rotation / student flow / traffic reduction

Raw idea: - Move students around campus in a planned way or reduce traffic locks by controlling movement.

Correct system interpretation: - Store building/room proximity or distance. - Penalize large movement between consecutive student classes. - Penalize high-traffic building transitions during peak times. - Optionally prefer same building or nearby rooms. - Do not attempt full transport simulation in MVP.

Morning classes

Raw idea: - Morning classes may build habits or save electricity in summer.

Correct system interpretation: - Make morning preference optional by department/program. - Do not force all students into early classes. - Treat as soft preference with configurable weight. - Allow season-specific or department-specific setting.

Early day ending

Raw idea: - End classes earlier to reduce fatigue and keep students fresh.

Correct system interpretation: - Add latest preferred end time. - Penalize classes after selected threshold. - Balance with room availability, teacher availability, and required weekly sessions.

Holidays and blocked days

Raw idea: - Add holidays and university closure days.

Correct system interpretation: - Calendar handling is a core requirement from day 1. - Holidays and blocked days are hard constraints. - If a holiday is added after publication, trigger re-optimization or manual repair.

Teacher custom changes without disturbing others

Raw idea: - Teacher timetable changes should not affect everyone else.

Correct system interpretation: - Use change request workflow. - Admin reviews and approves. - Re-optimization should preserve locked and accepted entries. - Minimize disturbance score. - Store new timetable version. - Show difference between old and revised timetable.

Near rooms or not always same room

Raw idea: - Classes should sometimes be close to each other, but not always same room depending on selected option.

Correct system interpretation: - Room proximity is a soft preference. - Room stability is another soft preference. - Room variety can be optional. - System should allow toggles and weights.

PART G — PROCESS MODEL ASSIGNMENT KNOWLEDGE

G1. Selected Model

Primary lifecycle: **Incremental Process Model**

Supporting technique: **Prototyping inside early increments**

Best wording: > The primary process model selected for the University Timetable Optimization and Management System is the Incremental Process Model. Prototyping is used as a supporting technique within selected early increments to validate screens, rule configuration, timetable grids, and reports.

Avoid weak wording: - Do not say only “Incremental Process Model with Prototyping” without clarification. - Do not make it sound like a random hybrid model.

G2. Why Incremental Fits This Project

Reasons: - The system is modular: data, rules, generation, review, repair, publishing. - Requirements may evolve after users see real timetable drafts. - Soft preference weights cannot be perfectly known before experimentation. - The solver/generation feature is risky and should be tested early. - Manual adjustment and re-optimization can be added later. - Each increment can be tested separately. - Coursework still needs formal documentation and traceability. - It is disciplined enough for an assignment but flexible enough for changing timetable rules.

G3. Why Waterfall Is Weak

Waterfall is less suitable because: - it assumes stable requirements; - timetable preferences are discovered after reviewing outputs; - teacher availability and holiday changes may occur late; - visual timetable review is needed before finalizing rules; - waiting until the end to test solver logic is risky; - late changes become expensive.

G4. Why Spiral Is Not the Best Main Model

Spiral is possible but not ideal because: - it is risk-driven and useful for large/high-risk systems; - it adds formal risk analysis overhead; - the project is a semester-scale student system; - the failure mode is a poor schedule, not a safety-critical disaster; - Incremental gives enough risk control without Spiral ceremony.

G5. Why Pure Agile/Scrum Is Not the Best Formal Label

Pure Agile/Scrum is not selected as the formal model because: - it may introduce sprint ceremony overhead; - academic reports often expect classic process model justification; - the core domain is stable enough for phased increments; - Agile practices can be used inside increments without naming Agile as the main lifecycle.

G6. Selection Criteria

Criteria used: - requirement volatility; - visual validation need; - technical uncertainty; - modular architecture; - documentation requirements; - semester-scale feasibility; - future extensibility; - need for user feedback; - testing after each working build; - ability to handle late changes.

G7. Increment Plan — Strong Version

Use this 6-increment plan:

1. **Master Data + Academic Calendar**
   - Admin can maintain teachers, courses, rooms, sections, timeslots, holidays, and blocked periods.
2. **Constraints & Preferences**
   - Admin can configure hard constraints and weighted soft preferences.
3. **First Working Timetable Generator**
   - Admin can generate a draft timetable and inspect conflicts or infeasibility.
4. **Review, Scoring, Manual Editing & Locking**
   - Admin can review quality, edit entries, and lock decisions.
5. **Repair / Re-optimization**
   - Admin can handle teacher/room/timing changes with minimal disruption.
6. **Publishing + Role Views + Reports**
   - Teachers and students view published timetables; Admin exports and reviews reports.

G8. Process Model Diagram Spec for Visio

Diagram title: **Figure 2.1 — Incremental Process Model**

Diagram structure: - Six increments left to right. - Each increment box has a small internal mini-cycle: Plan, Design, Build, Test, Review. - Horizontal arrows connect increments. - Top bracket spans all increments with label: **Documentation · Configuration Management · Continuous Testing** - Optional note: **Prototyping applied mainly in early increments for UI, rule setup, and timetable grid validation.**

Increment boxes: 1. Increment 1 — Master Data + Calendar 2. Increment 2 — Constraints & Preferences 3. Increment 3 — Generation + Conflicts 4. Increment 4 — Review + Manual Locking 5. Increment 5 — Repair / Re-optimization 6. Increment 6 — Publish + Reports

Correct meaning: - Development is controlled and phased. - Each increment adds user-visible value. - Feedback after each increment improves later increments. - Documentation and testing continue throughout. - Prototyping is a validation technique, not the main lifecycle label.

G9. Short Process Model Assignment Answer

The Incremental Process Model is selected because the timetable system can be developed in functional increments and because its requirements are likely to become clearer after users

review early timetable outputs. The system includes multiple modules such as academic data management, constraint configuration, generation, review, repair, and publishing. These modules can be developed step by step, and each increment can be tested before the next one begins. Prototyping is used inside early increments to validate screens and timetable views, but the primary lifecycle remains Incremental.

PART H — USE CASE KNOWLEDGE

H1. Compact Use Case Set for SRS Diagrams

Use these five if your teacher asks for simple UC-01 to UC-05:

- UC-01 Maintain Scheduling Data
- UC-02 Generate Timetable
- UC-03 Review and Adjust Timetable
- UC-04 Publish and View Timetable
- UC-05 Manage Change Requests and Re-optimize

This compact set is good for: - use case diagram; - system sequence diagram; - SRS requirement grouping; - simpler assignment submission.

H2. Expanded High-Level Use Case Set

Use these twelve if your teacher wants richer high-level use cases:

- UC01 Maintain Academic Calendar
- UC02 Manage Master Data
- UC03 Define Constraints and Preferences
- UC04 Generate Timetable
- UC05 Review Timetable Quality
- UC06 Request Teacher or Room Change
- UC07 Re-optimize Timetable After Change
- UC08 Lock Manual Decisions
- UC09 Publish Timetable
- UC10 View Personal Timetable
- UC11 View Resource Reports
- UC12 Manage Users and Roles

H3. Corrected Use Case Modelling Notes

Avoid these old weaknesses: - Do not separate holidays from academic calendar unless needed. Holidays belong under calendar management. - Do not let Teacher directly “re-optimize timetable.” Teacher requests a change. - Locking manual decisions may be either a use case or an included behavior under review/adjustment. For simple diagrams, keep it under Review and

Adjust. - Use <> and <> carefully. Many relationships are business dependencies, not UML include/extend. - A dependency such as “Generate Timetable depends on configured data and constraints” can be shown as a note rather than formal include.

H4. Use Case Diagram Actor Connections — Compact Version

System boundary: **University Timetable Optimization and Management System**

Actors outside: - Admin - Teacher - Student - Department Head

Use cases inside: - UC-01 Maintain Scheduling Data - UC-02 Generate Timetable - UC-03 Review and Adjust Timetable - UC-04 Publish and View Timetable - UC-05 Manage Change Requests and Re-optimize

Connections: - Admin → UC-01, UC-02, UC-03, UC-04, UC-05 - Department Head → UC-01, UC-03, UC-05 - Teacher → UC-04, UC-05 - Student → UC-04

Optional note relationships: - UC-02 depends on validated scheduling data and active rules. - UC-03 follows UC-02. - UC-04 requires an approved timetable version. - UC-05 may create a revised timetable version.

H5. Use Case Diagram Actor Connections — Expanded Version

Actors: - Timetable Administrator - Department Coordinator - Teacher - Student - Facility Manager - System Administrator

Use cases: - Maintain Academic Calendar - Manage Master Data - Define Constraints and Preferences - Generate Timetable - Review Timetable Quality - Request Teacher or Room Change - Re-optimize Timetable After Change - Lock Manual Decisions - Publish Timetable - View Personal Timetable - View Resource Reports - Manage Users and Roles

Connections: - Timetable Administrator → Maintain Academic Calendar, Manage Master Data, Define Constraints and Preferences, Generate Timetable, Review Timetable Quality, Re-optimize Timetable After Change, Lock Manual Decisions, Publish Timetable, View Resource Reports. - Department Coordinator → Manage Master Data, Define Constraints and Preferences, Review Timetable Quality, Request Teacher or Room Change. - Teacher → Request Teacher or Room Change, View Personal Timetable. - Student → View Personal Timetable. - Facility Manager → View Resource Reports. - System Administrator → Manage Users and Roles.

H6. Use Case Catalogue — Compact Version

UC-01 Maintain Scheduling Data

Actors: - Admin - Department Head

Essential goal: - Keep academic and resource data ready for scheduling.

Real meaning: - Admin enters courses, teachers, rooms, sections, timeslots, calendar dates, holidays, and blocked periods; Department Head may validate department-level data.

Preconditions: - Admin is logged in. - Admin has data-maintenance permission.

Main result: - Valid scheduling data exists for timetable generation.

UC-02 Generate Timetable

Actors: - Admin

Essential goal: - Produce a feasible timetable draft.

Real meaning: - Admin starts generation and the system assigns classes to timeslots and rooms while respecting hard constraints and optimizing selected soft preferences.

Preconditions: - Data exists. - Constraints exist. - Calendar and timeslots exist. - Input validation passes.

Main result: - A draft timetable, conflict report, score report, or infeasibility message is produced.

UC-03 Review and Adjust Timetable

Actors: - Admin - Department Head

Essential goal: - Assess and improve generated timetable quality.

Real meaning: - Admin and Department Head review timetable grid, conflicts, score breakdown, room usage, teacher load, and policy outcomes before acceptance.

Preconditions: - Draft timetable exists.

Main result: - Timetable is accepted, adjusted, locked, or sent for regeneration.

UC-04 Publish and View Timetable

Actors: - Admin - Teacher - Student

Essential goal: - Release an approved timetable and allow users to view it.

Real meaning: - Admin publishes approved version; teachers view teaching schedules; students view section schedules.

Preconditions: - Timetable version has been approved.

Main result: - Published timetable is visible to intended users.

UC-05 Manage Change Requests and Re-optimize

Actors: - Teacher - Admin - Department Head

Essential goal: - Handle changes after draft or publication without disturbing unrelated entries.

Real meaning: - Teacher or Department Head submits a change request; Admin reviews it; system re-optimizes while preserving locked and accepted entries where possible.

Preconditions: - Timetable entry exists. - Requester is authenticated.

Main result: - Request is approved/rejected/returned; if approved, revised timetable version is created.

PART I — FUNCTIONAL REQUIREMENTS

I1. Functional Requirements — Compact UC-01 to UC-05

UC-01 Maintain Scheduling Data

- FR-01.1 The system shall allow the Admin to add, update, view, and disable courses with course code, course name, credit hours, weekly session count, and department.
- FR-01.2 The system shall allow the Admin to maintain teacher records with name, department, contact information, teaching load, and availability status.
- FR-01.3 The system shall allow the Admin to maintain rooms with building, floor, capacity, room type, and available facilities such as lab equipment or projector.
- FR-01.4 The system shall allow the Admin to maintain sections/student groups with program, semester, student count, and assigned courses.
- FR-01.5 The system shall allow the Admin to define academic calendar dates, holidays, working days, and blocked periods.

UC-02 Generate Timetable

- FR-02.1 The system shall allow the Admin to start timetable generation for a selected academic term and department.
- FR-02.2 The system shall prevent two classes from being assigned to the same room at the same timeslot.
- FR-02.3 The system shall prevent a teacher from being assigned to more than one class at the same timeslot.
- FR-02.4 The system shall prevent a student section from being assigned to more than one class at the same timeslot.
- FR-02.5 The system shall generate a draft timetable using active hard constraints and selected soft preferences.
- FR-02.6 The system shall display generation progress, completion status, and failure messages if no feasible timetable can be found.

UC-03 Review and Adjust Timetable

- FR-03.1 The system shall show the generated timetable in day-wise and section-wise views.
- FR-03.2 The system shall display conflict counts, hard-constraint violations, soft-preference penalties, and room utilization summaries.
- FR-03.3 The system shall allow the Admin to manually adjust selected timetable entries when permitted.
- FR-03.4 The system shall allow the Admin to lock selected classes so that later re-optimization does not change them.

- FR-03.5 The system shall validate every manual change before saving it.

UC-04 Publish and View Timetable

- FR-04.1 The system shall allow the Admin to publish only an approved timetable version.
- FR-04.2 The system shall allow Teachers to view their assigned teaching timetable.
- FR-04.3 The system shall allow Students to view the published timetable for their section, semester, or group.
- FR-04.4 The system shall allow users to filter the published timetable by day, course, teacher, room, section, or department.
- FR-04.5 The system shall support exporting the published timetable in printable or downloadable format.

UC-05 Manage Change Requests and Re-optimize

- FR-05.1 The system shall allow Teachers to submit timetable change requests with reason, affected class, preferred alternative time, and urgency.
- FR-05.2 The system shall allow the Department Head or Admin to approve, reject, or return change requests for clarification.
- FR-05.3 The system shall allow the Admin to re-optimize the timetable after an approved change request while preserving locked and accepted entries where possible.
- FR-05.4 The system shall store each re-optimized timetable as a new version.
- FR-05.5 The system shall show the difference between the previous timetable version and the revised version before publication.

I2. Functional Requirements — Expanded UC01 to UC12

UC01 Maintain Academic Calendar

- The system shall allow the Timetable Administrator to add semester start and end dates.
- The system shall allow the Timetable Administrator to add holidays, blackout dates, and special closure days.
- The system shall prevent classes from being scheduled on holidays or unavailable dates.

UC02 Manage Master Data

- The system shall allow the Timetable Administrator to add, update, and delete teachers.
- The system shall allow the Timetable Administrator to add, update, and delete courses, sections, rooms, and timeslots.
- The system shall store room details such as room number, capacity, building, floor, and available facilities.

UC03 Define Constraints and Preferences

- The system shall allow the Timetable Administrator to define hard constraints such as teacher clashes, room clashes, and section clashes.
- The system shall allow the Timetable Administrator to define soft preferences such as early ending, nearby rooms, morning classes, and energy-saving options.

- The system shall allow preferences to be enabled, disabled, or given priority according to university needs.

UC04 Generate Timetable

- The system shall generate a timetable based on teachers, courses, rooms, sections, timeslots, holidays, and constraints.
- The system shall prevent two teachers, rooms, or student sections from being assigned to conflicting classes at the same time.
- The system shall assign rooms according to class size, room capacity, and room availability.

UC05 Review Timetable Quality

- The system shall display the generated timetable in a clear weekly view.
- The system shall show conflicts, warnings, and violated preferences in the generated timetable.
- The system shall show timetable quality details such as room usage, teacher load, and number of clashes.

UC06 Request Teacher or Room Change

- The system shall allow a teacher to submit a timetable change request.
- The system shall allow the Department Coordinator to request changes for teachers, rooms, or class timings.
- The system shall store the reason and status of each change request.

UC07 Re-optimize Timetable After Change

- The system shall allow the Timetable Administrator to re-optimize the timetable after a teacher, room, or timing change.
- The system shall try to minimize disturbance to already scheduled classes during re-optimization.
- The system shall keep locked classes unchanged during re-optimization.

UC08 Lock Manual Decisions

- The system shall allow the Timetable Administrator to lock specific classes, rooms, or timeslots.
- The system shall prevent locked timetable entries from being changed automatically.
- The system shall allow the Timetable Administrator to unlock a timetable entry when needed.

UC09 Publish Timetable

- The system shall allow the Timetable Administrator to publish the final approved timetable.
- The system shall make the published timetable visible to teachers and students.
- The system shall allow the final timetable to be exported or printed.

UC10 View Personal Timetable

- The system shall allow teachers to view their own weekly timetable.
- The system shall allow students to view their section timetable.
- The system shall display class day, time, room, course, and teacher details in the timetable.

UC11 View Resource Reports

- The system shall allow the Facility Manager or Admin to view room utilization reports.
- The system shall show free, used, underused, and overused rooms.
- The system shall show capacity warnings and peak load periods.

UC12 Manage Users and Roles

- The system shall allow the System Administrator to create and manage user accounts.
- The system shall assign users to roles such as Admin, Teacher, Student, Department Head, Facility Manager, and System Administrator.
- The system shall restrict system features according to user role.

PART J — NON-FUNCTIONAL REQUIREMENTS

J1. Performance

- The system shall open common pages such as login, dashboard, and timetable view within 3 seconds under normal university network conditions.
- The system shall save ordinary data-entry forms within 2 seconds under normal load.
- The system shall generate a small departmental timetable draft within 5 minutes for typical semester data.
- The system shall show a progress indicator if timetable generation takes more than 10 seconds.
- The system shall allow the Admin to cancel or retry generation if the solver cannot produce a result within the configured limit.
- For a prototype dataset, generation should ideally complete within 30 seconds to 5 minutes depending on problem size and algorithm.

J2. Security

- The system shall require authenticated login for all administrative, teacher, and department-head functions.
- The system shall restrict timetable creation, generation, approval, and publishing to authorized Admin users only.
- The system shall allow Teachers to view only their own timetable and submit only permitted change requests.
- The system shall allow Students to view only published timetable information, not draft versions or administrative controls.

- The system shall record audit logs for timetable generation, manual edits, approvals, publication, and change-request decisions.
- The system shall prevent unauthorized access to restricted screens and API actions.

J3. Usability

- The system shall provide clear labels, validation messages, and simple navigation for non-technical users.
- The system shall display timetable information in familiar grid form by day and time.
- The system shall show conflicts and rule violations in plain language, not only numeric scores.
- The system shall allow constraint weights and preferences to be configured through forms rather than through source-code changes.
- The system shall provide search and filtering for teachers, courses, rooms, sections, and days.
- The system shall avoid forcing users to understand solver internals.

J4. Availability

- The system shall be available during university working hours for timetable administration and timetable viewing.
- The system shall keep the last published timetable available even if a new draft generation fails.
- The system shall protect published timetable versions from accidental deletion.
- The system shall support database backup so that timetable data can be restored if needed.
- Teachers and students should always be able to access the last published timetable during the semester.

PART K — PROJECT CONSTRAINTS AND ASSUMPTIONS

K1. Project Constraints

Hardware: - The system should run on ordinary university lab PCs or a standard cloud/server machine.

Platform: - The user interface should work in common modern browsers.

Budget: - The first version should use free or student-accessible tools where possible.

Time: - The project must fit a semester-scale software engineering course.

Data: - The quality of timetable output depends on complete and correct teacher, room, course, section, and calendar data.

Algorithm: - Advanced optimization research is postponed to a later technical phase.

Integration: - Full university ERP integration is not required for the first version.

K2. Assumptions

- The first implementation targets one department or faculty rather than the entire university.
- Teachers and sections follow a weekly recurring timetable pattern.
- The Admin has authority to publish the final timetable.
- The Department Head can review or approve changes according to local policy.
- Holidays and blocked periods are known before generation or can be added later through re-optimization.
- The solver may return a feasible but not perfect timetable.
- Admin can review, adjust, lock, and improve timetable entries manually.
- Users have basic internet or local network access.
- University data is available in some form.

PART L — DATA FLOW DIAGRAM KNOWLEDGE

L1. Level 0 Context Diagram

Correct representation: - One process only: **University Timetable Optimization System** - External entities outside: - Admin - Teacher - Student - Department Head - Facility Manager - Data flows: - Admin → System: academic data, constraints, generation command, approval decision - System → Admin: draft timetable, conflict report, score report, publish status - Teacher → System: availability, change request - System → Teacher: personal timetable, request status - Student → System: timetable view request - System → Student: published section timetable - Department Head → System: review feedback, approval/change request - System → Department Head: quality summary, draft timetable, reports - Facility Manager → System: room data/availability if included - System → Facility Manager: room utilization report

Rule: - Do not show database or solver as external entities in Level 0 context diagram. - Keep system as black box.

L2. Level 1 DFD

Processes: 1. Maintain Scheduling Data 2. Configure Rules and Preferences 3. Generate Timetable 4. Review and Adjust Timetable 5. Publish Timetable and Views 6. Manage Change Requests and Re-optimization

Data stores: - D1 Academic Data Store - D2 Rules and Constraints Store - D3 Timetable Versions Store - D4 Change Request Store - Optional D5 User and Role Store - Optional D6 Audit Log Store

External entities: - Admin - Teacher - Student - Department Head - Facility Manager

Key flows: - Admin sends course/teacher/room/section/calendar data to Process 1. - Process 1 writes Academic Data Store. - Admin sends hard constraints and soft preferences to Process 2. -

Process 2 writes Rules and Constraints Store. - Process 3 reads Academic Data Store and Rules Store. - Process 3 writes draft timetable to Timetable Versions Store. - Process 4 reads draft timetable and returns review results. - Admin edits/locks entries through Process 4. - Process 5 publishes approved timetable. - Teacher/Student read published views through Process 5. - Teacher/Department Head submits change request through Process 6. - Process 6 writes Change Request Store. - Approved change request triggers re-optimization and writes new timetable version.

L3. Level 2 DFD for Generate Timetable

Process 3 decomposition: 3.1 Load Scheduling Data 3.2 Validate Data Completeness 3.3 Build Problem Model 3.4 Apply Hard Constraints 3.5 Apply Soft Preferences and Weights 3.6 Run Solver / Assignment Engine 3.7 Check Conflicts and Score Timetable 3.8 Save Draft Timetable Version 3.9 Return Result to Admin

Important flows: - D1 Academic Data → 3.1 - D2 Rules/Constraints → 3.3/3.4/3.5 - 3.2 invalid data → Admin error list - 3.6 infeasible result → Admin infeasibility report - 3.7 score and conflict results → Admin review - 3.8 saved draft → D3 Timetable Versions

---

PART M — SYSTEM SEQUENCE DIAGRAM KNOWLEDGE

M1. System Sequence Diagram Selected Use Case

Use case: **UC-02 Generate Timetable** or **UC04 Generate Timetable**

Abstraction rule: - Only show external actor and system. - Do not show internal database, backend classes, solver object, or UI components. - Time flows top to bottom.

Lifelines: - Admin - University Timetable Optimization System

M2. SSD Main Flow

Messages: 1. Admin → System: selectGenerateTimetable() 2. System → Admin: displayGenerationSettings() 3. Admin → System: chooseTermDepartmentAndPreferences() 4. System → Admin: showValidationSummary() 5. Admin → System: confirmGeneration() 6. System → Admin: showProgressIndicator() 7. System → System: validateDataAndRunGeneration() 8. System → Admin: returnDraftTimetable() 9. System → Admin: displayConflictAndScoreReport() 10. Admin → System: saveDraftVersion() 11. System → Admin: confirmDraftSaved()

Alternate flow: - If required data is missing: - System → Admin: showMissingDataErrors() - Admin → System: updateDataOrCancel() - If no feasible timetable can be found: - System → Admin: showInfeasibilityReport() - Admin → System: adjustConstraintsOrRetry() - If generation is long: - System → Admin: showProgressOrTimeoutWarning()

M3. Visual Rule

Use arrows: - solid arrow from Admin to System for action/request; - dashed or solid return arrow from System to Admin for response; - activation bars optional; - no database boxes in system sequence diagram.

PART N — ARCHITECTURE DIAGRAM KNOWLEDGE

N1. Four Core Components

1. Frontend
2. Backend API
3. Database
4. Solver Engine

N2. Component Responsibilities

Frontend: - login screen; - admin dashboard; - data-entry forms; - constraint configuration screens; - timetable grid; - conflict report UI; - teacher view; - student view; - change request form; - report pages.

Backend API: - authentication; - authorization; - validation; - business workflow; - data persistence; - versioning; - audit logging; - solver orchestration; - export/report endpoints.

Database: - users and roles; - teachers; - courses; - rooms; - sections; - timeslots; - academic calendar; - constraints; - timetable versions; - timetable entries; - change requests; - locks; - audit logs.

Solver Engine: - builds assignment problem; - applies hard constraints; - applies soft preference weights; - produces room/timeslot assignments; - returns infeasible classes; - returns score and penalty report; - supports later replacement with OR-Tools, Timefold, OptaPlanner-style solver, or custom heuristic.

N3. Architecture Diagram Flows

- Frontend → Backend API: HTTPS/REST request
- Backend API → Frontend: JSON response / rendered result
- Backend API → Database: SQL queries / writes
- Database → Backend API: records/results
- Backend API → Solver Engine: problem model
- Solver Engine → Backend API: assignment result + score + conflicts
- Backend API → Database: save timetable version
- Backend API → Frontend: timetable grid + reports

N4. Architecture Correctness Notes

- Users are not architecture components.

- Database is not an actor.
- Solver is internal component, not external actor in use case diagram.
- Frontend should not directly access database.
- Frontend should not directly access solver engine in a clean design.
- Backend API should control access, validation, and orchestration.

PART O — SWIMLANE WORKFLOW KNOWLEDGE

O1. Swimlane Diagram Title

Figure 5.4 — Swimlane Diagram: Timetable Generation Workflow

O2. Swimlanes

Three lanes: 1. Admin 2. System 3. Teacher / Student

O3. Swimlane Main Flow

1. Admin: Start
2. Admin: Enter / update master data
   - Courses, teachers, rooms, sections, timeslots
3. Admin: Define constraints and preferences
   - Hard constraints, soft goals, holidays, availability
4. System: Validate input data
5. System decision: Data valid?
   - No → Admin: Correct data / rules → back to validation
   - Yes → continue
6. System: Build timetable model
7. System: Generate timetable
8. System: Check conflicts and score quality
9. Admin: Review generated timetable
10. Admin decision: Acceptable?

- No → Admin: Adjust constraints or lock decisions → System rebuilds/generates again
- Yes → Admin: Approve and publish timetable

11. System: Store version and publish
12. Teacher/Student: View personal timetable
13. Teacher/Student decision: Change needed?

- No → End
- Yes → Submit change request

14. System: Analyze impact and re-optimize
15. Admin: Review revised timetable
16. Admin decision: Approve revision?

- No → Adjust constraints or lock decisions → re-optimize
- Yes → System: Update published timetable

17. Teacher/Student: Receive updated timetable
18. End

O4. Swimlane Correctness Notes

- Admin enters data, defines rules, reviews, approves.
- System validates, generates, scores, stores, publishes, re-optimizes.
- Teacher/Student only view and request change; they do not directly generate timetable.
- Decision diamonds must have labeled Yes/No paths.
- Cross-lane arrows show handoffs.
- Loops show revision/re-optimization.
- End states can appear after no change or after updated timetable.

PART P — DOMAIN / DATA MODEL KNOWLEDGE

P1. Minimum Database Schema

Users

Fields: - user_id - name - email - password_hash - role_id - department_id - status - created_at

Roles

Fields: - role_id - role_name - permissions

Departments

Fields: - department_id - department_name - head_user_id

Programs

Fields: - program_id - department_id - program_name

Academic Terms

Fields: - term_id - term_name - semester_start - semester_end - status

Academic Calendar

Fields: - calendar_id - term_id - working_days - default_start_time - default_end_time

Holidays

Fields: - holiday_id - calendar_id - holiday_date - description - type

Courses

Fields: - course_id - course_code - course_name - credit_hours - weekly_session_count - required_room_type - department_id

Teachers

Fields: - teacher_id - user_id - department_id - teaching_load_limit - status

Teacher Availability

Fields: - availability_id - teacher_id - day - start_time - end_time - availability_type - reason

Sections

Fields: - section_id - program_id - semester - section_name - student_count

Rooms

Fields: - room_id - building_id - room_number - floor - capacity - room_type - facilities - status

Buildings

Fields: - building_id - building_name - campus_zone - coordinate_x - coordinate_y

Timeslots

Fields: - timeslot_id - day - start_time - end_time - slot_type

Constraint Rules

Fields: - constraint_id - constraint_name - constraint_type - description - is_active - is_hard - priority_weight

Timetable Versions

Fields: - version_id - term_id - department_id - status - created_by - created_at - published_at - total_score

Timetable Entries

Fields: - entry_id - version_id - course_id - teacher_id - section_id - room_id - timeslot_id - is_locked - lock_reason - status

Change Requests

Fields: - request_id - entry_id - requested_by - request_type - reason - preferred_day - preferred_timeslot_id - urgency - status - decision_by - decision_note - created_at - decided_at

Audit Logs

Fields: - audit_id - user_id - action_type - entity_type - entity_id - old_value - new_value - timestamp

P2. Timetable Entry Is the Central Entity

The most important scheduled object is TimetableEntry. It connects: - course/class; - teacher; - student section; - room; - timeslot; - timetable version; - lock status.

Every timetable generation, manual edit, publication, and re-optimization is ultimately about creating or changing TimetableEntry records.

PART Q — REPORTS AND QUALITY METRICS

Q1. Hard Constraint Metrics

Track: - teacher clashes; - room clashes; - section clashes; - holiday violations; - capacity violations; - room suitability violations; - locked-entry violations; - unavailable-teacher assignments.

Target: - hard constraint violations should be zero in a feasible published timetable.

Q2. Soft Preference Metrics

Track: - early ending score; - morning preference score; - room proximity score; - room stability score; - teacher compactness score; - student compactness score; - building/energy compaction score; - movement/traffic score; - minimal disruption score; - free day or half-day score if selected.

Q3. Operational Reports

Reports: - room utilization report; - teacher load report; - section load report; - conflict summary; - unplaced class list; - soft penalty breakdown; - late-day class count; - building movement report; - change impact report; - timetable version comparison report.

PART R — ACCEPTANCE TEST IDEAS

R1. Basic Data Tests

AT-01: - Scenario: Admin adds a course with valid name, code, weekly sessions, and department. - Expected: Course is saved and appears in scheduling data.

AT-02: - Scenario: Admin adds a room with capacity and room type. - Expected: Room is available for scheduling if active.

AT-03: - Scenario: Admin adds a holiday. - Expected: No generated class appears on that holiday.

R2. Generation Tests

AT-04: - Scenario: Admin generates timetable with complete valid data. - Expected: Draft timetable is produced.

AT-05: - Scenario: Two classes require same teacher in same timeslot. - Expected: System prevents clash or selects another timeslot.

AT-06: - Scenario: No room is large enough for a section. - Expected: System reports infeasibility or capacity warning.

R3. Manual Edit Tests

AT-07: - Scenario: Admin manually moves a class to occupied room/timeslot. - Expected: System rejects change or shows clear conflict warning.

AT-08: - Scenario: Admin locks a timetable entry and re-optimizes. - Expected: Locked entry remains unchanged.

R4. Publishing Tests

AT-09: - Scenario: Admin publishes approved timetable. - Expected: Teacher and student views show the new timetable.

AT-10: - Scenario: Student tries to view draft timetable. - Expected: Student sees only published timetable.

R5. Change Request Tests

AT-11: - Scenario: Teacher submits change request. - Expected: Request is stored with status and timestamp.

AT-12: - Scenario: Approved change request triggers re-optimization. - Expected: New timetable version is created, and unrelated locked entries are preserved.

PART S — RISKS AND MITIGATION

S1. Incorrect Input Data

Risk: - Timetable generation fails or produces poor schedule.

Mitigation: - Add validation, duplicate checks, missing-field warnings, and data review before generation.

S2. Too Many Hard Constraints

Risk: - No feasible timetable can be generated.

Mitigation: - Show infeasibility reasons. - Allow Admin to convert some rules into soft preferences if policy allows.

S3. Unclear Actor Permissions

Risk: - Users may access wrong functions.

Mitigation: - Define role-based access control early. - Test each role separately.

S4. Solver Complexity

Risk: - Generation may be slow or poor.

Mitigation: - Start with transparent MVP solver. - Keep solver replaceable. - Later upgrade to OR-Tools/Timefold/custom optimization.

S5. Diagram Misrepresentation

Risk: - Examiner may penalize wrong UML/DFD boundaries.

Mitigation: - Use correct system boundary. - Keep actors outside use case diagram. - Keep database out of context diagram. - Avoid incorrect include/extend relationships. - Use notes for business dependencies.

S6. Over-ambitious Scope

Risk: - Project becomes too broad.

Mitigation: - Keep MVP to one department/faculty. - Exclude exam timetabling, buses, hostels, ERP, and individualized student optimization.

PART T — IMPLEMENTATION ROADMAP

T1. Immediate Next Tasks

1. Finalize database schema.
2. Build master data screens for courses, teachers, rooms, sections, timeslots, and calendar.
3. Build constraint configuration screen.
4. Build first timetable grid view.
5. Implement basic clash detection.
6. Implement first generator MVP.
7. Add conflict report and quality score.
8. Add manual edit and locking.
9. Add change request workflow.
10. Add re-optimization/versioning.
11. Add publish and role-based timetable views.

T2. Priority Order

1. Database schema design.
2. Master data UI.
3. Academic calendar and holidays.
4. Constraint configuration UI.
5. Basic clash validator.
6. First working generator.
7. Review/conflict report.
8. Manual locking.
9. Re-optimization.
10. Publishing and views.

T3. Critical Milestone

Most critical milestone: **First Working Timetable Generator**

Reason: - It exposes algorithmic and modelling risks early. - It reveals missing constraints. - It gives real feedback from timetable outputs. - It validates data structure and rule model.

PART U — VISIO PROMPTS AND DRAWING GUIDES

U1. Process Model Visio Prompt

Create a professional software engineering process model diagram for a project named "University Timetable Optimization System." Use an Incremental Process Model layout. Show six increments from left to right: Master Data + Academic Calendar, Constraints & Preferences, First Working Timetable Generator, Review + Manual Locking, Repair / Re-optimization, and Publish + Reports. Inside or below each increment show a mini-cycle: Plan, Design/Prototype, Build, Test, Review. Connect increments with right arrows. Add a long bracket across the top labelled "Documentation · Configuration Management · Continuous Testing." Make the style formal, clean, and academic.

U2. Use Case Diagram Visio Prompt — Compact

Create a UML Use Case Diagram for "University Timetable Optimization and Management System." Draw one system boundary rectangle with that name. Outside the boundary place four actors: Admin, Teacher, Student, Department Head. Inside the boundary draw five ovals: UC-01 Maintain Scheduling Data, UC-02 Generate Timetable, UC-03 Review and Adjust Timetable, UC-04 Publish and View Timetable, UC-05 Manage Change Requests and Re-optimize. Connect Admin to all five use cases. Connect Department Head to Maintain Scheduling Data, Review and Adjust Timetable, and Manage Change Requests and Re-optimize. Connect Teacher to Publish and View Timetable and Manage Change Requests and Re-optimize. Connect Student to Publish and View Timetable. Use clean UML notation. Avoid unnecessary include/extend arrows; if needed, use notes for dependencies.

U3. System Sequence Diagram Visio Prompt

Create a System Sequence Diagram for UC-02 Generate Timetable. Use two lifelines: Admin on the left and University Timetable Optimization System on the right. Show messages top to bottom: selectGenerateTimetable(), displayGenerationSettings(), chooseTermDepartmentAndPreferences(), showValidationSummary(), confirmGeneration(), showProgressIndicator(), validateDataAndRunGeneration(), returnDraftTimetable(), displayConflictAndScoreReport(), saveDraftVersion(), confirmDraftSaved(). Add alternate notes for missing data and infeasible timetable. Do not show database, solver classes, backend classes, or UI components separately because this is a system-level sequence diagram.

U4. Architecture Diagram Visio Prompt

Create a four-component architecture diagram for a web-based University Timetable Optimization System. Show four boxes: Frontend, Backend API, Database, Solver Engine. Connect Frontend to Backend API with "HTTPS/REST." Connect Backend API to Database with "SQL queries / records." Connect Backend API to Solver Engine with "Problem model." Connect Solver Engine back to Backend API with "Assignment result + score + conflicts." Connect Backend API back to Frontend with "Timetable grid + reports." Add small labels inside each component describing responsibility. Keep the diagram clean and formal.

U5. Swimlane Diagram Visio Prompt

Create a swimlane diagram titled "Figure 5.4 — Swimlane Diagram: Timetable Generation Workflow." Use three vertical swimlanes: Admin, System, Teacher / Student. Admin starts, enters master data, defines constraints and preferences, reviews generated timetable, accepts or adjusts rules, approves and publishes, reviews revised timetable, and approves revision. System validates input data, checks if data is valid, builds timetable model, generates timetable, checks conflicts and scores quality, stores version and publishes, analyzes impact and re-optimizes, and updates published timetable. Teacher/Student views personal timetable, decides if change is needed, submits change request, receives updated timetable, and ends. Use rounded rectangles, decision diamonds, clear Yes/No labels, and cross-lane arrows.

U6. Context Diagram Visio Prompt

Create a DFD Level 0 Context Diagram for "University Timetable Optimization and Management System." Draw one central process circle/rounded rectangle labelled with the system name. Place external entities outside: Admin, Teacher, Student, Department Head, Facility Manager. Show high-level data flows: Admin sends scheduling data, constraints, generation commands, approvals; System returns draft timetable, conflict report, quality score, and publish status. Teacher sends availability/change requests and receives personal timetable/request status. Student requests timetable and receives published section timetable. Department Head sends review feedback/approval and receives quality reports. Facility Manager sends room data and receives room utilization reports. Do not show database or solver in Level 0.

U7. Level 1 DFD Visio Prompt

Create a Level 1 DFD for the University Timetable Optimization and Management System. Show processes: 1.0 Maintain Scheduling Data, 2.0 Configure Rules and Preferences, 3.0 Generate Timetable, 4.0 Review and Adjust Timetable, 5.0 Publish Timetable and Views, 6.0 Manage Change Requests and Re-optimization. Show data stores: D1 Academic Data, D2 Rules and Constraints, D3 Timetable Versions, D4 Change Requests, D5 Users and Roles, D6 Audit Logs. Show external entities: Admin, Teacher, Student, Department Head, Facility Manager. Connect data flows logically between actors, processes, and data stores.

PART V — SOURCE-STYLE RAW NOTES

V1. Teacher Assignment Pattern

Teacher-style assignments expected: - decide on a process model; - justify why that model fits the project; - create high-level use cases; - include diagram descriptions; - draw diagrams in Visio; - keep use cases essential and real.

V2. Original Missing Diagram List

Missing/needed diagrams: - Figure 2.1 Incremental Process Model - Figure 5.4 Swimlane Diagram - Figure 4.1 Use Case Diagram - Figure 5.1 Level 0 Context Diagram - Figure 5.2 Level 1 DFD - Figure 5.3 Level 2 DFD for Generate Timetable - Figure 5.5 System Sequence Diagram for Generate Timetable / UC04 - Figure 5.6 Architecture Diagram

V3. Main Diagram Deliverables

Most important Visio deliverables: 1. Incremental Process Model 2. Swimlane Diagram

Other useful diagrams: - Use Case Diagram - Context Diagram - Level 1 DFD - Level 2 DFD - System Sequence Diagram - Architecture Diagram - Domain/Data Model

V4. Key Examiner-Proof Modelling Corrections

- Say Incremental is the primary model; prototyping is supporting technique.
- Do not misuse UML include/extend for simple preconditions.
- Actor-role boundaries must be clear.
- Teacher requests change; Admin performs re-optimization.
- Holiday management belongs to academic calendar.
- Manual locking can be part of review/adjustment if using compact use cases.
- System boundary must contain only system functions.
- External actors must remain outside.
- Internal components belong in architecture diagram, not use case diagram.
- Database belongs in DFD data store/architecture, not as a human actor.
- Context diagram should show the system as one black box.

- SSD should show actor-system messages only.

PART W — RAW EXTRACTED TABLES FROM THE CURRENT SRS DOCUMENT

The following tables were extracted from the existing SRS-style document and preserved as raw knowledge.

Source Table 1

Item	Details
Project	University Timetable Optimization and Management System
Document Purpose	To define scope, requirements, actors, process model, use cases, data flows, and visual design diagrams for a university timetable optimization website.
Audience	Course instructor, examiner, student project team, and future developers.
Version	1.0
Status	Draft for academic submission and review.

Source Table 2

User	Main Need	System Support
Admin / Timetable Administrator	Create, review, repair, and publish timetables.	Full access to data, constraints, generation, review, adjustment, versioning, and publication.
Teacher	View teaching timetable and request changes when needed.	Personal timetable view, availability/preference submission, and change-request form.
Student	View official section or group timetable.	Published timetable view filtered by section, semester, course, or day.
Department Head	Review policy choices and approve important changes.	Read-only review, approval support, and quality reports.
Future Facility Manager	Monitor room/resource usage.	Resource reports and building/room utilization summaries if included in a later version.

Source Table 3

Criterion	Why it matters for this project
Requirement volatility	Timetable rules and preferences often become clearer only after users see draft schedules.
Visual validation	Users need to see forms, timetable grids, conflict reports, and review screens early.
Technical uncertainty	Generation, re-optimization, and minimal-disruption repair introduce risk.
Modular architecture	The system can be developed in slices: data, rules, generation, review, repair, publication.
Coursework documentation	The project needs formal deliverables, diagrams, and traceability.
Semester-scale feasibility	The lifecycle must be disciplined but not too heavy for a student project.

Source Table 4

Model	Fit	Reason
Waterfall	Low	Best for stable requirements; weak for a timetable system where hidden constraints appear after draft review.
Incremental	High	Builds useful modules in controlled slices and allows later increments to absorb feedback from earlier ones.
Spiral	Medium	Strong for high-risk projects but too management-heavy for a semester-scale academic project.
Agile/XP practices	Medium	Useful for short feedback loops inside increments, but too broad as the formal lifecycle label for this report.

Source Table 5

Term	Meaning
Admin	The timetable administrator who operates the scheduling system.
Section	A student group/cohort taking a set of courses in a semester.
Timeslot	A defined day and time range in which a

Term	Meaning
	class can be scheduled.
Hard Constraint	A rule that must not be violated, such as no teacher clash or no room double-booking.
Soft Preference	A desirable goal that can be violated with penalty, such as early finish or room proximity.
Solver Engine	The optimization component that assigns classes to rooms and timeslots.
Timetable Version	A saved draft or published copy of a timetable.
Re-optimization	The process of repairing an existing timetable while minimizing unnecessary changes.

Source Table 6

User	Expected Skill Level	Implication for Design
Admin	Moderate computer literacy and timetable knowledge.	Needs clear forms, validation errors, preview screens, and editable rule configuration.
Teacher	Basic web use.	Needs simple view, availability/preference entry, and change-request submission.
Student	Basic web/mobile use.	Needs fast access to a readable timetable without complex settings.
Department Head	Review-oriented user.	Needs summaries, quality scores, approvals, and policy visibility.

Source Table 7

Field	Detail
Function	Maintain Scheduling Data
Description	Admin records academic and resource data required for scheduling.
Input	Courses, teachers, rooms, sections, timeslots, calendar.
Source	Admin / Department Head
Output	Stored scheduling data.
Destination	Database
Requires	Authorized login and valid form data.

Field	Detail
Pre-condition	Admin is logged in.
Post-condition	Data becomes available for generation.
Side Effects	Invalid data may block generation until corrected.

Source Table 8

Field	Detail
Function	Generate Timetable
Description	System creates a timetable draft from data, constraints, and preferences.
Input	Term, department, constraints, weights, active data.
Source	Admin
Output	Draft timetable, score, conflict report.
Destination	Admin / Database
Requires	Scheduling data and constraints.
Pre-condition	Required data exists and validation passes.
Post-condition	Draft version is stored for review.
Side Effects	High computation load during generation.

Source Table 9

Field	Detail
Function	Review and Adjust Timetable
Description	Admin reviews draft quality and edits or locks selected entries.
Input	Draft timetable, conflicts, manual changes.
Source	Admin / Department Head
Output	Adjusted timetable version.
Destination	Database
Requires	Existing draft timetable.
Pre-condition	Draft has been generated.
Post-condition	Revised version is saved or marked for publication.
Side Effects	Changes may require re-validation.

Source Table 10

Field	Detail
Function	Publish Timetable
Description	Approved timetable is released to teachers

Field	Detail
	and students.
Input	Approved version ID.
Source	Admin
Output	Published timetable views and export files.
Destination	Teachers / Students
Requires	Reviewed and approved timetable.
Pre-condition	Timetable status is approved.
Post-condition	Users can view official timetable.
Side Effects	Previous version becomes archived.

Source Table 11

Field	Detail
Function	Manage Change Request
Description	Teacher or department submits and tracks schedule change request.
Input	Affected class, reason, preferred time, urgency.
Source	Teacher / Department Head
Output	Approved/rejected request and possible revised timetable.
Destination	Admin / Database
Requires	Published or draft timetable.
Pre-condition	Requester is authenticated.
Post-condition	Request is logged and acted upon.
Side Effects	May trigger re-optimization and notifications.

Source Table 12

Constraint Type	Constraint
Hardware	The system should run on ordinary university lab PCs or a standard cloud/server machine.
Platform	The user interface should work in common modern browsers.
Budget	The first version should use free or student-accessible tools where possible.
Deadline	The scope must remain achievable within a semester project timeline.
Database	A relational database should be used for

Constraint Type	Constraint
	structured academic records and timetable versions.
Algorithm	The SRS defines solver requirements but does not finalize the optimization algorithm.
Data	Accurate teacher, room, course, section, and calendar data must be supplied before reliable generation is possible.

Source Table 13

ID	Use Case	Actors	Type	Essential Goal	Real Meaning
UC-01	Maintain Scheduling Data	Admin, Department Head	Primary	Keep academic and resource data ready for scheduling.	The Admin enters courses, teachers, rooms, sections, timeslots, calendar, and holidays; the Department Head may validate policy-sensitive data.
UC-02	Generate Timetable	Admin	Primary	Produce a feasible timetable draft.	The Admin starts generation and the system assigns classes to timeslots and rooms while respecting constraints.

ID	Use Case	Actors	Type	Essential Goal	Real Meaning
UC-03	Review and Adjust Timetable	Admin, Department Head	Primary	Assess and improve generated timetable quality.	The Admin and Department Head review conflicts, scores, room usage, and policy outcomes before acceptance.
UC-04	Publish / View Timetable	Admin, Teacher, Student	Primary	Release and access the official timetable.	The Admin publishes the approved version; teachers and students view filtered official timetables.
UC-05	Manage Change Requests	Teacher, Admin, Department Head	Primary	Submit, approve, and handle timetable changes.	A teacher requests a change; the Department Head/Admin reviews it; the Admin may re-optimize with minimum disruption.

Source Table 14

Field	Detail
Actors	Admin, Department Head
Purpose	To maintain the academic and resource information required before timetable generation.
Pre-condition	Admin is logged in and has data-maintenance permission.
Post-condition	Valid academic, room, section, teacher,

Field	Detail
	calendar, and timeslot data are stored.
Main Flow	Admin opens the scheduling data module; system displays forms; Admin adds or updates records; system validates required fields and uniqueness; system saves records; Department Head may review policy-sensitive data.
Alternative Flow	If data is incomplete or invalid, the system displays field-level errors and prevents saving until corrected.

Source Table 15

Field	Detail
Actors	Admin
Purpose	To create a feasible timetable draft from configured data and rules.
Pre-condition	Scheduling data, constraints, calendar, and timeslots exist and pass validation.
Post-condition	A draft timetable version is generated or a clear failure report is produced.
Main Flow	Admin selects term and department; system loads active data and constraints; Admin confirms generation settings; system validates inputs; system builds the problem model; solver creates candidate assignment; system stores draft version; system shows timetable, score, and conflicts.
Alternative Flow	If data is missing or infeasible, the system stops generation and shows reasons such as missing room, teacher clash, or impossible availability constraint.

Source Table 16

Field	Detail
Actors	Admin, Department Head
Purpose	To evaluate timetable quality and make controlled improvements.
Pre-condition	A draft timetable exists.
Post-condition	Draft is accepted, revised, locked partially, or sent back for re-generation.
Main Flow	Admin opens generated draft; system

Field	Detail
	displays timetable grid and quality report; Admin reviews conflicts and soft penalties; Department Head reviews policy outcomes; Admin edits or locks selected entries; system validates changes and stores a revised version.
Alternative Flow	If a manual adjustment creates a clash, the system rejects it or marks it as unresolved until corrected.

Source Table 17

Field	Detail
Actors	Admin, Teacher, Student
Purpose	To publish the approved timetable and make it available to users.
Pre-condition	A timetable version has been reviewed and approved.
Post-condition	Teachers and students can view the official published timetable.
Main Flow	Admin selects approved version; system asks for confirmation; Admin publishes; system marks version as published; teachers and students log in or open view page; system shows filtered timetable.
Alternative Flow	If no approved version exists, the system prevents publication and shows a warning.

Source Table 18

Field	Detail
Actors	Teacher, Admin, Department Head
Purpose	To handle timetable changes after a draft or published timetable exists.
Pre-condition	Requester is authenticated and a timetable entry exists.
Post-condition	Request is approved, rejected, returned, or used to create a revised timetable version.
Main Flow	Teacher submits request with reason and affected class; system stores request; Department Head/Admin reviews request; Admin chooses re-optimization or manual adjustment; system tries to preserve locked entries and minimize unnecessary changes;

Field	Detail
	revised version is reviewed before publication.
Alternative Flow	If the requested change is impossible, the system shows impact reasons and the request can be rejected or revised.

Source Table 19

Use Case	Linked FR IDs	Verification Focus	Related Diagrams
UC-01 Maintain Scheduling Data	FR-01 to FR-05	Create/update records; validate required fields and duplicates.	Context, DFD, Use Case
UC-02 Generate Timetable	FR-06 to FR-11	Generate draft; prevent teacher, room, and section clashes.	Use Case, SSD, Architecture
UC-03 Review and Adjust Timetable	FR-12 to FR-16	Show conflicts, score quality, validate manual edits, lock entries.	Use Case, Swimlane, DFD
UC-04 Publish and View Timetable	FR-17 to FR-21	Publish approved version; allow teacher/student filtered views and export.	Use Case, Architecture
UC-05 Manage Change Requests	FR-22 to FR-26	Submit, approve/reject, re-optimize, version, and compare changes.	Use Case, Swimlane, Domain Model

Source Table 20

Test ID	Scenario	Expected Result
AT-01	Admin adds a course with valid name, code, weekly sessions, and department.	Course is saved and appears in scheduling data.
AT-02	Admin tries to generate timetable while a teacher has two classes at the same timeslot.	System prevents clash in generated timetable or reports infeasibility.

Test ID	Scenario	Expected Result
AT-03	Admin manually moves a class to an occupied room and timeslot.	System rejects the change or shows a clear conflict warning.
AT-04	Teacher submits a change request for an assigned class.	Request is stored and visible to Admin/Department Head.
AT-05	Admin publishes an approved timetable.	Teacher and Student views show the published version, not draft versions.
AT-06	Admin re-optimizes after a teacher change request with locked entries.	Locked entries remain unchanged and a new version is created.

Source Table 21

Risk	Impact	Mitigation
Incorrect input data	Timetable generation fails or produces poor results.	Add validation, required fields, duplicate checks, and data review before generation.
Too many hard constraints	No feasible timetable can be generated.	Show infeasibility reasons and allow the Admin to relax selected rules if they are actually soft preferences.
Unclear actor permissions	Users may access wrong functions.	Define role-based access control early and test each role separately.
Algorithm complexity	Generation may take too long.	Start with a small department and set solver time limits.
Late teacher/room changes	Published timetable becomes outdated.	Use versioning, change requests, locking, and re-optimization.

PART X — RAW EXTRACTED PARAGRAPH TEXT FROM THE CURRENT SRS DOCUMENT

This section preserves paragraph-level source material from the current SRS-style document.

[X001] Software Requirements Specification (SRS) University Timetable Optimization and Management System

[X002] Prepared for: Software Engineering Subject Document type: Project Proposal + Process Model + SRS + Use Cases + Diagrams

[X003] Prepared by: Student Name / Registration No. Submission date: _____

[X004] Version: 1.0

[X005] Document Control

[X006] Table of Contents

[X021] 1. Project Proposal

[X022] 1.1 Project Name

[X023] University Timetable Optimization and Management System

[X024] 1.2 What is this system?

[X025] The proposed system is a web-based platform that helps a university or department prepare weekly class timetables for teachers, student sections, courses, rooms, and time slots. Instead of manually placing every class into a room and time, the administrator enters academic data, defines scheduling rules, selects policy preferences, and asks the system to generate a timetable draft. The system then checks conflicts, scores timetable quality, supports review and adjustment, and publishes the approved timetable for teachers and students.

[X026] 1.3 What problem does it solve right now?

[X027] The system solves the immediate problem of creating a clash-free and practical university timetable when there are multiple teachers, rooms, classes, student groups, holidays, and policy restrictions. It reduces manual scheduling effort, prevents common clashes, makes timetable quality visible, and supports controlled changes without rebuilding the entire schedule from zero.

[X028] 1.4 Objectives

[X029] Reduce manual workload in university timetable preparation.

[X030] Prevent teacher, room, section, and timeslot clashes before publication.

[X031] Allow administrators to configure hard constraints and soft preferences without editing source code.

[X032] Support institutional goals such as compact schedules, room proximity, early day ending, energy-aware grouping, and reduced campus movement where selected.

[X033] Support timetable repair after teacher leave, room unavailability, holidays, or approved change requests.

[X034] Provide teachers and students with filtered published timetable views.

[X035] Maintain versions, audit trail, and clear reports for administrative review.

[X036] 1.5 Explicitly Not in Scope

[X037] Exam timetabling is not included in the first version.

[X038] Individual personalized timetable optimization for every student is not included; the first version schedules by section/group/cohort.

[X039] Transport routing, bus scheduling, hostel allocation, and parking optimization are not included.

[X040] Full multi-campus simulation is not included; building and room proximity may be represented only through simple distance/penalty values.

[X041] Automatic replacement of academic policy decisions is not included; the system supports configuration, but final approval remains with the administrator or department head.

[X042] Deep algorithm comparison and solver benchmarking are not included in this SRS; algorithm design can be handled in a later technical document.

[X043] 1.6 Main Features

[X044] Academic calendar, holidays, and blocked-period management.

[X045] Course, teacher, section, room, building, and timeslot management.

[X046] Teacher availability and room-suitability recording.

[X047] Hard-constraint and soft-preference configuration.

[X048] Timetable generation from configured data and rules.

[X049] Conflict detection and timetable quality scoring.

[X050] Manual adjustment, locking, and versioned re-optimization.

[X051] Teacher or department change-request workflow.

[X052] Published timetable views for teachers and students.

[X053] Reports for room usage, daily load, late classes, compactness, and resource utilization.

[X054] Export to PDF/Excel/CSV for academic use.

[X055] 1.7 Users and Needs

[X056] 1.8 System Dependencies

[X057] A database is required to store users, academic data, constraints, timetable versions, and change requests.

[X058] Internet or local network access is required for web-based use.

[X059] A backend API is required to connect the frontend, database, and solver engine.

[X060] A solver/optimization module is required for generating candidate timetables.

[X061] Authentication is required so that only authorized users can change schedules.

[X062] Optional email or notification service may be used for publishing updates and change-request notifications.

[X063] 2. Process Model Selection

[X064] The selected primary lifecycle model is the Incremental Process Model. Prototyping is used only as a supporting validation technique inside early increments, especially for screens, rule configuration, timetable grids, and reports. This avoids the academic weakness of inventing an unclear hybrid model: the process model is Incremental, while prototyping is a technique used within selected increments.

[X065] 2.1 Selection Criteria

[X066] 2.2 Candidate Models Considered

[X067] 2.3 Chosen Model Justification

[X068] The system is modular: calendar/data setup, constraints, generation, review, repair, and publishing can be built separately.

[X069] The first generated timetable may reveal missing rules, so feedback must be absorbed without restarting the entire project.

[X070] The generation MVP should appear early enough to expose algorithmic and data-model risks.

[X071] Re-optimization after change requests is a natural later increment, not an afterthought.

[X072] Documentation remains possible because each increment produces updated requirements, test cases, and design artifacts.

[X073] Figure 1. Incremental process model with internal mini-cycles and continuous documentation/testing.

[X074] 2.4 Diagram Description

[X075] Figure 1 shows a baseline requirements and architecture stage followed by five functional increments. Each increment contains a small internal cycle of planning, design/prototyping, building, testing, and review. The top band shows activities that continue throughout development: documentation, configuration management, verification, and continuous testing. This representation is academically safer because it keeps Incremental as the primary lifecycle and shows prototyping as a validation technique rather than as a separate mixed lifecycle.

[X076] 3. Requirements Specification

[X077] 3.1 Purpose

[X078] The purpose of this SRS is to define the functional and non-functional requirements of the University Timetable Optimization and Management System. The document gives enough detail for software engineering analysis, design diagrams, use cases, and later implementation planning, while avoiding deep algorithm design at this stage.

[X079] 3.2 Scope

[X080] The first version supports weekly course/class timetabling for one university department or faculty. It manages academic data, constraints, timetable generation, review, repair after changes, and publication. The system handles teachers, courses, sections, rooms, time slots, academic calendar, holidays, and timetable versions.

[X081] 3.3 Definitions and Abbreviations

[X082] 3.4 Product Perspective

[X083] The system is a web application with a frontend interface, backend API, relational database, and solver engine. The frontend is used by administrators, teachers, students, and department heads. The backend enforces validation, permissions, workflow, and integration with the solver. The database stores academic records and timetable versions. The solver engine produces candidate assignments from the problem model sent by the backend.

[X084] 3.5 User Characteristics

[X085] 3.6 General Constraints

[X086] The system should be developed within a semester-scale student project timeline.

[X087] The first version should use a simple web architecture rather than an enterprise-scale multi-campus platform.

[X088] The system should work on normal university lab computers and common browsers.

[X089] The solver should be replaceable later without rewriting the entire frontend.

[X090] The final report should remain understandable to a software engineering examiner, not only to optimization experts.

[X091] 4. Functional Requirements

[X092] Every requirement in this section uses the required format: “The system shall …”. The requirements are grouped by high-level use case.

[X093] UC-01 Maintain Scheduling Data

[X094] The system shall allow the Admin to add, update, view, and disable courses with course code, course name, credit hours, weekly session count, and department.

[X095] The system shall allow the Admin to maintain teacher records with name, department, contact information, teaching load, and availability status.

[X096] The system shall allow the Admin to maintain rooms with building, floor, capacity, room type, and available facilities such as lab equipment or projector.

[X097] The system shall allow the Admin to maintain sections/student groups with program, semester, student count, and assigned courses.

[X098] The system shall allow the Admin to define academic calendar dates, holidays, working days, and blocked periods.

[X099] UC-02 Generate Timetable

[X100] The system shall allow the Admin to start timetable generation for a selected academic term and department.

[X101] The system shall prevent two classes from being assigned to the same room at the same timeslot.

[X102] The system shall prevent a teacher from being assigned to more than one class at the same timeslot.

[X103] The system shall prevent a student section from being assigned to more than one class at the same timeslot.

[X104] The system shall generate a draft timetable using active hard constraints and selected soft preferences.

[X105] The system shall display generation progress, completion status, and failure messages if no feasible timetable can be found.

[X106] UC-03 Review and Adjust Timetable

[X107] The system shall show the generated timetable in day-wise and section-wise views.

[X108] The system shall display conflict counts, hard-constraint violations, soft-preference penalties, and room utilization summaries.

[X109] The system shall allow the Admin to manually adjust selected timetable entries when permitted.

[X110] The system shall allow the Admin to lock selected classes so that later re-optimization does not change them.

[X111] The system shall validate every manual change before saving it.

[X112] UC-04 Publish and View Timetable

[X113] The system shall allow the Admin to publish only an approved timetable version.

[X114] The system shall allow Teachers to view their assigned teaching timetable.

[X115] The system shall allow Students to view the published timetable for their section, semester, or group.

[X116] The system shall allow users to filter the published timetable by day, course, teacher, room, section, or department.

[X117] The system shall support exporting the published timetable in printable or downloadable format.

[X118] UC-05 Manage Change Requests and Re-optimize

[X119] The system shall allow Teachers to submit timetable change requests with reason, affected class, preferred alternative time, and urgency.

[X120] The system shall allow the Department Head or Admin to approve, reject, or return change requests for clarification.

[X121] The system shall allow the Admin to re-optimize the timetable after an approved change request while preserving locked and accepted entries where possible.

[X122] The system shall store each re-optimized timetable as a new version.

[X123] The system shall show the difference between the previous timetable version and the revised version before publication.

[X124] 4.6 Level 3 SRS Function Table

[X125] Level 3 SRS - Maintain Scheduling Data

[X126] Level 3 SRS - Generate Timetable

[X127] Level 3 SRS - Review and Adjust Timetable

[X128] Level 3 SRS - Publish Timetable

[X129] Level 3 SRS - Manage Change Request

[X130] 5. Non-Functional Requirements

[X131] 5.1 Performance

[X132] The system shall open common pages such as login, dashboard, and timetable view within 3 seconds under normal university network conditions.

[X133] The system shall save ordinary data-entry forms within 2 seconds under normal load.

[X134] The system shall generate a small departmental timetable draft within 5 minutes for typical semester data.

[X135] The system shall show a progress indicator if timetable generation takes more than 10 seconds.

[X136] The system shall allow the Admin to cancel or retry generation if the solver cannot produce a result within the configured limit.

[X137] 5.2 Security

[X138] The system shall require authenticated login for all administrative, teacher, and department-head functions.

[X139] The system shall restrict timetable creation, generation, approval, and publishing to authorized Admin users only.

[X140] The system shall allow Teachers to view only their own timetable and submit only permitted change requests.

[X141] The system shall allow Students to view only published timetable information, not draft versions or administrative controls.

[X142] The system shall record audit logs for timetable generation, manual edits, approvals, publication, and change-request decisions.

[X143] 5.3 Usability

[X144] The system shall provide clear labels, validation messages, and simple navigation for non-technical users.

[X145] The system shall display timetable information in familiar grid form by day and time.

[X146] The system shall show conflicts and rule violations in plain language, not only as numeric scores.

[X147] The system shall allow constraint weights and preferences to be configured through forms rather than through source-code changes.

[X148] The system shall provide search and filtering for teachers, courses, rooms, sections, and days.

[X149] 5.4 Availability

[X150] The system shall be available during university working hours for timetable administration and timetable viewing.

[X151] The system shall keep the last published timetable available even if a new draft generation fails.

[X152] The system shall protect published timetable versions from accidental deletion.

[X153] The system shall support database backup so that timetable data can be restored if needed.

[X154] 6. Constraints and Assumptions

[X155] 6.1 Project Constraints

[X156] 6.2 Assumptions

[X157] The first implementation targets one department or faculty rather than the entire university.

[X158] Teachers and sections follow a weekly recurring timetable pattern.

[X159] The Admin has authority to publish the final timetable.

[X160] The Department Head can review or approve changes according to local policy.

[X161] Holidays and blocked periods are known before generation or can be added later through re-optimization.

[X162] The solver may return a feasible but not perfect timetable; the Admin can review, adjust, and improve it.

[X163] 7. Data Flow and Architecture

[X164] Figure 2. Level 0 context diagram.

[X165] Figure 2 shows the system as one black-box boundary. This is the correct representation for a context diagram: external actors are outside the system and interact with the system through high-level data or commands.

[X166] Figure 3. Level 1 data flow diagram.

[X167] Figure 3 decomposes the black-box system into the main data processes: maintaining scheduling data, configuring rules, generating timetables, reviewing/adjusting drafts, and publishing views. It also identifies the main data stores: academic data, rules/preferences, timetable versions, and change requests.

[X168] Figure 4. Architecture diagram with four core components.

[X169] Figure 4 shows the implementation architecture. The frontend communicates with the backend API through HTTPS/REST. The backend reads and writes persistent data through SQL queries. It sends a problem model to the solver engine and receives assignment results. The solver is an internal service/module, not a direct user-facing component.

[X170] 8. Use Cases

[X171] Figure 5. UML use case diagram for UC-01 through UC-05.

[X172] 8.1 Diagram Description

[X173] Figure 5 contains one clear system boundary labelled University Timetable Optimization System. Actors remain outside the boundary. Use cases remain inside the boundary. Direct lines show actor participation. Dashed arrows are labelled as business dependencies/preconditions, not as UML include/extend relationships, because generation depends on configured data and publishing depends on approval, but these are not reusable sub-use cases executed inside another use case.

[X174] 8.2 Use Case Catalogue

[X175] 9. Expanded Use Cases

[X176] UC-01 Maintain Scheduling Data

[X177] UC-02 Generate Timetable

[X178] UC-03 Review and Adjust Timetable

[X179] UC-04 Publish / View Timetable

[X180] UC-05 Manage Change Requests

[X181] 10. System Sequence Diagram

[X182] Figure 6. System sequence diagram for UC-02 Generate Timetable.

[X183] Figure 6 shows only the interaction between the external actor and the system for UC-02. It deliberately does not show internal classes, database tables, or solver internals. Time flows from top to bottom. This is the correct abstraction for a system sequence diagram.

[X184] 11. Workflow and Domain Model

[X185] Figure 7. Swimlane workflow for timetable generation and revision.

[X186] Figure 7 separates responsibilities across Admin, System, and Teacher/Student lanes. The Admin enters data, defines rules, reviews drafts, and approves revisions. The System validates, generates, scores, stores, publishes, and re-optimizes. Teachers and students view timetables and submit change requests when needed.

[X187] Figure 8. Conceptual domain/data entity model.

[X188] Figure 8 identifies the main data concepts that the system must store or process. The most important object is TimetableEntry, which connects course, teacher, section, room, and timeslot under rules and calendar restrictions. TimetableVersion and ChangeRequest support review, publication, and repair after changes.

[X189] 12. Traceability and Test Planning

[X190] 12.1 Requirement Traceability Matrix

[X191] 12.2 Acceptance Test Ideas

[X192] 12.3 Risks and Mitigation

[X193] 13. References

[X194] Sommerville, I. (2016). Software Engineering (10th ed.). Pearson.

[X195] Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach. McGraw-Hill.

[X196] Larman, C. (2004). Applying UML and Patterns (3rd ed.). Prentice Hall.

[X197] Booch, G., Rumbaugh, J., & Jacobson, I. (2005). The Unified Modeling Language User Guide (2nd ed.). Addison-Wesley.

[X198] Di Gaspero, L., McCollum, B., & Schaerf, A. (2007). Curriculum-Based Course Timetabling, International Timetabling Competition Track 3 specification.

[X199] UniTime. University Timetabling and Course Management documentation.

[X200] Google OR-Tools. Scheduling and CP-SAT solver documentation.

[X201] Timefold Solver. School timetabling constraints and hard/soft score documentation.

[X202] Appendix A. Visio Drawing Guide

[X203] A.1 Use Case Diagram Visio Instructions

[X204] Create a UML Use Case Diagram. Draw one system boundary rectangle labelled "University Timetable Optimization System". Place Admin, Teacher, Student, and Department Head outside the boundary. Inside the boundary draw five ovals: UC-01 Maintain Scheduling Data, UC-02 Generate Timetable, UC-03 Review & Adjust Timetable, UC-04 Publish / View Timetable, and UC-05 Manage Change Requests. Connect actors only to the use cases they participate in. Do not place actors inside the system boundary. Avoid questionable include/extend relations; use notes for business dependencies if needed.

[X205] A.2 System Sequence Diagram Visio Instructions

[X206] Create two lifelines: Admin on the left and System on the right. Use horizontal arrows from Admin to System for user actions and arrows from System to Admin for responses. Show the UC-02 flow: select Generate Timetable, display settings, confirm term/data/rules, validate input, show progress/errors, run generation, return draft timetable, save draft, and open review screen. Do not include database or solver as lifelines if the assignment asks for a simple system sequence diagram with only user and system.

[X207] A.3 Architecture Diagram Visio Instructions

[X208] Draw four component boxes: Frontend, Backend API, Database, and Solver Engine. Connect Frontend to Backend API with "HTTPS/REST". Connect Backend API to Database with "SQL queries" and "records/results". Connect Backend API to Solver Engine with "Problem

model” and Solver Engine back to Backend API with “Assignment result”. Add a note that users do not directly access the solver engine.

END OF KNOWLEDGE BASE

Recommended next use: - Paste sections A–K into a requirements report. - Paste sections G and U1 into Assignment 1. - Paste sections H, I, and U2 into Assignment 2. - Paste sections L–O and U3–U7 when building diagrams in Visio. - Use sections P–T when starting actual implementation.

31. Raw Appendix: Existing SRS.md Baseline

The following appendix preserves the uploaded SRS.md baseline.

Software Requirements Specification

Project: University Timetable Optimization and Management System
Version: 0.1 rough build baseline
Date: 2026-04-26

1. Introduction

1.1 Purpose

This SRS defines the first buildable baseline for a web-based University Timetable Optimization and Management System. It consolidates the project material found in the repository, especially REAL/questioning the customers.docx, the combined assignment documents, the use-case diagrams, DFD diagrams, sequence diagram, and architecture sketches.

1.2 Product Scope

The system helps a university department create, review, repair, publish, and report weekly class timetables. It manages teachers, courses, rooms, sections, timeslots, holidays, constraints, and soft scheduling preferences. The first version targets one department or limited university scope.

1.3 Problem Statement

Manual timetable creation is slow, error-prone, and difficult to update when teacher availability, room capacity, holidays, sections, and institutional preferences change. The system shall reduce scheduling clashes, expose timetable quality, and support controlled changes without disturbing unrelated timetable entries.

1.4 In Scope

- Manage teachers, rooms, courses, sections, timeslots, academic calendar, and holidays.
- Define hard constraints and soft preferences.
- Generate an initial clash-free timetable when feasible.
- Report conflicts, unplaced classes, room use, teacher load, and soft-score penalties.
- Allow manual locking and future re-optimization.
- Publish views for teachers and students.
- Export or print the final timetable in a later increment.

1.5 Out of Scope

- Exam timetabling.
- Bus, hostel, attendance, fee, or full ERP management.
- Personalized timetables for individual students.
- Advanced AI prediction.
- Multi-campus deployment in the first version.

2. Overall Description

2.1 Users and Needs

- Timetable Administrator: maintain data, define rules, generate schedules, repair clashes, lock entries, approve, and publish.
- Department Coordinator: validate department data, review generated schedules, request changes.
- Teacher: view personal timetable, submit availability or change requests.
- Student: view section timetable.
- Facility Manager: review room use, free rooms, capacity issues, and building load.
- System Administrator: manage accounts, roles, settings, backups, and security controls.

2.2 Development Lifecycle

The selected lifecycle is the Incremental Process Model with early prototyping as a supporting technique. Planned increments are:

1. Master data and resource setup.
2. Constraints and policy configuration.
3. Initial timetable generation MVP.
4. Review, repair, and controlled re-optimization.
5. Publication and operational reporting.

2.3 Architecture

The planned architecture has four separable components:

- Frontend: HTML/CSS/JavaScript screens for operators and viewers.
- Backend API: validation, persistence, role-aware actions, and orchestration.
- Database: persistent master data, constraints, timetable versions, and audit records.

- Solver Engine: transforms data into a timetable assignment and score report.

The current implementation uses a dependency-light Python backend, SQLite database, and modular vanilla frontend. The architecture intentionally keeps the solver swappable so a future OR-Tools CP-SAT implementation can replace the current transparent MVP solver.

3. Functional Requirements

UC01 - Maintain Academic Calendar

- FR-01.1 The system shall allow the Timetable Administrator to add semester dates.
- FR-01.2 The system shall allow the Timetable Administrator to add holidays, blackout dates, and closure days.
- FR-01.3 The system shall prevent generated classes from being placed on holiday or blocked timeslots.

UC02 - Manage Master Data

- FR-02.1 The system shall allow the Timetable Administrator to add, update, and delete teachers.
- FR-02.2 The system shall allow the Timetable Administrator to add, update, and delete courses, sections, rooms, and timeslots.
- FR-02.3 The system shall store room number, capacity, building, floor, and facilities.
- FR-02.4 The system shall store course teacher, section, weekly session count, and required room type.

UC03 - Define Constraints and Preferences

- FR-03.1 The system shall support hard constraints for teacher clashes, room clashes, section clashes, room capacity, holidays, and teacher availability.
- FR-03.2 The system shall support soft preferences for morning classes, early endings, nearby rooms, energy-aware building compaction, and traffic-reduction modes.
- FR-03.3 The system shall allow preferences to be enabled, disabled, and weighted.

UC04 - Generate Timetable

- FR-04.1 The system shall generate a weekly timetable from teachers, courses, rooms, sections, timeslots, holidays, and constraints.
- FR-04.2 The system shall avoid assigning the same teacher, section, or room to two classes in the same timeslot.
- FR-04.3 The system shall assign rooms according to section size, room capacity, and room availability.
- FR-04.4 The system shall return unplaced classes when a complete timetable is infeasible.
- FR-04.5 The system shall calculate a timetable quality score and soft-constraint penalty.

UC05 - Review Timetable Quality

- FR-05.1 The system shall display the generated timetable in a weekly grid.
- FR-05.2 The system shall show conflicts, warnings, and unplaced classes.
- FR-05.3 The system shall show room utilization, teacher load, and total penalty.

UC06 - Request Timetable Change

- FR-06.1 The system shall allow teachers to submit change requests.
- FR-06.2 The system shall allow Department Coordinators to request changes for teachers, rooms, or class timings.
- FR-06.3 The system shall store each request reason, status, requester, and timestamp.

UC07 - Re-optimize Timetable After Change

- FR-07.1 The system shall allow the Timetable Administrator to re-optimize after a teacher, room, or timing change.
- FR-07.2 The system shall minimize disturbance to already accepted classes.
- FR-07.3 The system shall keep locked timetable entries unchanged.

UC08 - Lock Manual Decisions

- FR-08.1 The system shall allow the Timetable Administrator to lock specific timetable entries.
- FR-08.2 The system shall prevent locked entries from being changed automatically.
- FR-08.3 The system shall allow authorized unlocking.

UC09 - Publish Timetable

- FR-09.1 The system shall allow the Timetable Administrator to publish an approved timetable.
- FR-09.2 The system shall make the published timetable visible to teachers and students.
- FR-09.3 The system shall support export or printing in a later increment.

UC10 - View Personal Timetable

- FR-10.1 The system shall allow teachers to view their weekly timetable.
- FR-10.2 The system shall allow students to view their section timetable.
- FR-10.3 The system shall display day, time, room, course, teacher, and section.

UC11 - View Resource Reports

- FR-11.1 The system shall show room utilization reports.
- FR-11.2 The system shall identify free, used, underused, and overused rooms.
- FR-11.3 The system shall show capacity warnings and peak load periods.

UC12 - Manage Users and Roles

- FR-12.1 The system shall store user accounts and roles.
- FR-12.2 The system shall restrict timetable management actions to authorized roles.
- FR-12.3 The system shall prevent unauthorized access to restricted screens and API actions in production.

4. Non-Functional Requirements

4.1 Performance

- NFR-01 Normal dashboard and data requests should respond within 2 seconds for department-scale data.

- NFR-02 Initial timetable generation should complete within 30 seconds for the first academic prototype dataset.
- NFR-03 The solver shall expose partial or infeasible results rather than failing silently.

4.2 Security

- NFR-04 Role-based access shall separate Administrator, Coordinator, Teacher, Student, Facility Manager, and System Administrator capabilities.
- NFR-05 The system shall validate input before database writes.
- NFR-06 Production deployment shall use HTTPS, password hashing, audit logs, and database backups.

4.3 Usability

- NFR-07 The timetable shall be presented in a weekly grid readable by non-technical users.
- NFR-08 Conflicts and warnings shall be visible next to the generated timetable.
- NFR-09 Master data and policy screens shall use consistent labels and compact workflows suitable for repeated administrative use.

4.4 Availability and Reliability

- NFR-10 The system shall persist timetable data in a database so page refreshes and logouts do not lose work.
- NFR-11 The system shall maintain timetable versions so generated drafts can be reviewed later.
- NFR-12 The system shall fail gracefully when no feasible assignment exists.

4.5 Maintainability

- NFR-13 Frontend, backend API, repository, service, database, and solver modules shall be separated.
- NFR-14 The solver interface shall allow replacement by CP-SAT or another optimizer without rewriting the UI.
- NFR-15 Configuration data and seed data shall be isolated from algorithm logic.

5. Data Requirements

Core entities:

- User(id, name, email, role)
- Teacher(id, name, department, max_daily_load)
- Room(id, code, building, floor, capacity, features)
- Section(id, name, department, size)
- Course(id, code, title, teacher_id, section_id, weekly_sessions, required_room_type)
- Timeslot(id, day, start_time, end_time, sort_order, is_morning, is_last_slot)
- Holiday(id, name, day)
- TeacherAvailability(id, teacher_id, timeslot_id, is_available)
- Preference(id, key, label, enabled, weight, value)

- TimetableVersion(id, name, status, score, hard_conflicts, soft_penalty, created_at)
- TimetableEntry(id, version_id, course_id, teacher_id, section_id, room_id, timeslot_id, locked, status)
- ChangeRequest(id, requester_id, target_type, target_id, reason, status, created_at)

6. Solver Foundation

The first build models timetabling as assignment of course events to timeslots and rooms under hard and soft constraints. This follows the course timetabling literature, where a feasible timetable satisfies hard constraints and soft-constraint violations become penalties to minimize.

Current MVP algorithm:

1. Expand each course into weekly session events.
2. Sort events by constrainedness: larger sections, fewer available teacher slots, and higher session pressure first.
3. For each event, rank feasible room-timeslot candidates by soft penalty.
4. Use bounded backtracking to assign candidates while checking hard constraints.
5. Save placed entries and report unplaced entries with a distance-to-feasibility score.
6. Return soft penalties for last-slot usage, non-morning placement when morning preference is enabled, late-day placement, and building movement.

This is a transparent constructive/backtracking baseline. It is not claimed to be globally optimal. A future increment can replace it with CP-SAT, simulated annealing, tabu search, or a hybrid optimizer behind the same solver interface.

Research and technical references:

- Andrea Bettinelli, Valentina Cacchiani, Roberto Roberti, and Paolo Toth, “An overview of curriculum-based course timetabling”, TOP, 2015. https://link.springer.com/article/10.1007/s11750-015-0366-z
- Rhydian Lewis, Ben Paechter, and Barry McCollum, “Post Enrolment based Course Timetabling: A Description of the Problem Model used for Track Two of the Second International Timetabling Competition”, 2007. https://rhydlewis.eu/papers/PostEnrolTechRep.pdf
- Andrea Schaerf, “A Survey of Automated Timetabling”, Artificial Intelligence Review, 1999. https://www.scirp.org/reference/referencespapers?referenceid=1938970
- Google OR-Tools CP-SAT documentation for future solver replacement. https://developers.google.com/optimization/cp/cp_solver

7. Constraints

- The first implementation uses SQLite for local development.
- The first implementation supports a single department-scale dataset.
- Authentication is represented in data and UI shape but is not production-enforced yet.
- The solver is a research-grounded MVP baseline, not a full industrial optimizer.

8. Acceptance Criteria For Current Build

- The app starts locally from a single Python command.
- The database schema initializes automatically.
- Seed data appears in the dashboard.
- The user can trigger timetable generation.
- The generated timetable is persisted as a version.
- The frontend displays timetable entries, warnings, room use, and teacher load.
- The codebase is modular enough for independent frontend, backend, database, and solver changes.